Institut Supérieur
des Technologies de l'Information
et de la Communication



ARCHITECTURE IOT

Enseignantes : Meriam Dhouibi & Afef Saidi

Niveau : 3ème Année SE-IoT

Année universitaire : 2025 - 2026

Plan

Chapitre 1 : Introduction à l'IoT

Chapitre 2 : La partie embarquée des objets connectés

Chapitre 3 : Plateformes IoT

Chapitre 4 : Cloud Computing

Chapitre 5 : Node-RED

Chapitre 6 : Les protocoles de communication pour l'IoT

Chapitre 7 : REST API

Chapitre I

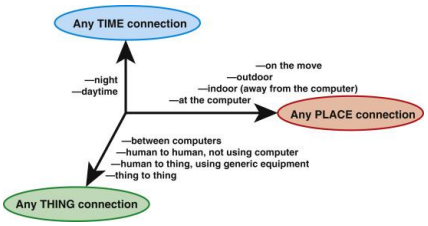
Introduction à l'IoT

Introduction à l'IoT

l'IoT ?

• **IoT :** un réseau massif de dispositifs physiques équipés de capteurs, de logiciels, d'électronique et d'un réseau qui permet aux dispositifs d'échanger ou de collecter des données et d'effectuer certaines actions.

⟹ Les deux aspects de l'IoT « temporel et spatial » permettent aux personnes de se connecter de n'importe où et à n'importe quel moment à travers des objets connectés.



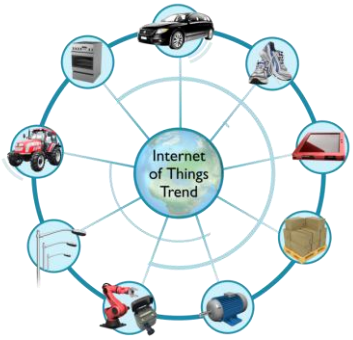
4

Introduction à l'IoT

Applications

Les applications de l'IoT touchent pratiquement aujourd'hui toute la vie quotidienne :

- La santé et les systèmes de télésurveillance.
- L'agriculture connectée pour optimiser l'usage de l'eau.
- Les véhicules connectés aident à optimiser la gestion du trafic urbain.
- Les appareils électroménagers connectés aident à optimiser la consommation et la distribution de l'énergie électrique.
- Montres connectées pour le bien-être et le sport.
- Etc.



5

Introduction à l'IoT

Solution IoT

- Une solution IoT est plus qu'un système embarqué doté d'une fonction de connectivité.
- Une solution IoT est un ensemble d'appareils et de capteurs connectés à une plateforme Cloud via une passerelle (Gateway).
- La plateforme Cloud fournit l'infrastructure nécessaire pour gérer, stocker, sécuriser et analyser une grande quantité de données afin d'extraire des informations précieuses.



6

Introduction à l'IoT

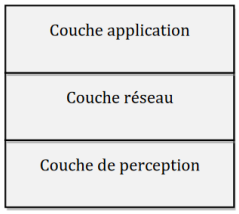
Architecture de l'IoT

- L'architecture d'une solution IoT varie d'un système à l'autre en se basant sur le type de la solution à mettre en place.
- L'architecture la plus élémentaire est une architecture à trois couches:

– **La couche perception** possède des capteurs et actionneurs qui détectent et recueillent des informations sur l'environnement.

– **La couche réseau** est responsable de la connexion, du transport et du traitement des données issues des capteurs et actionneurs.

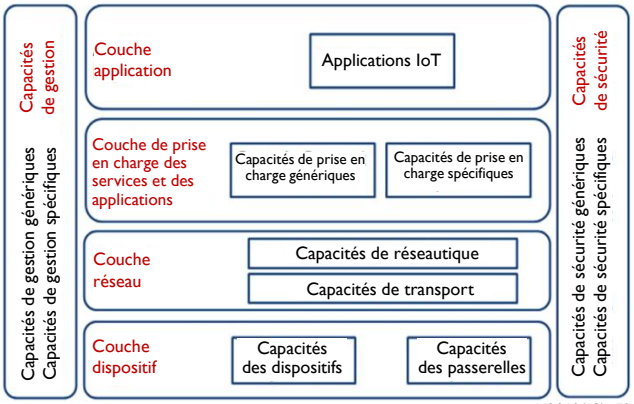
– **La couche application** est chargée de fournir à l'utilisateur des services spécifiques et applications intelligentes.



7

Introduction à l'IoT

Architecture IoT de référence



Y.2060(12)_F04

8

Introduction à l'IoT

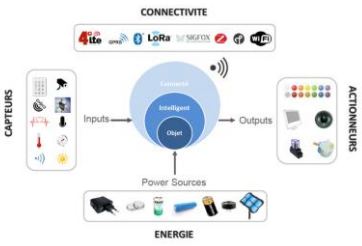
Composantes d'une solution IoT

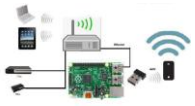
Généralement, une solution IoT est formée des composants suivants :

1. **Capteurs d'entrée** pour mesurer les variables d'environnement physique nécessaires.
2. **Actionneurs de sortie** pour contrôler l'environnement physique, tels que les objets en mouvement et l'augmentation de la température, etc…
3. **Le périphérique de connexion réseau** pour permettre à l'appareil de partager ses données collectées avec Cloud.

Cet appareil peut faire partie de la carte IoT comme les cartes Raspberry Pi ou d'une carte d'extension externe comme les cartes Arduino.

4. **La carte IoT** collecte les signaux des capteurs, contrôle les actionneurs et envoie les données collectées et l'état de l'appareil au réseau.





Gateway à Base de carte Raspberry Pi



Gateway à Base de carte Arduino

9

Introduction à l'IoT

Capteurs

Capteur : C'est un dispositif utilisé pour détecter un événement ou une grandeur physique, tels que luminosité, température, humidité du sol, pression, etc. et qui fournit un signal électrique correspondant.

- Les capteurs IoT sont généralement de petite taille, ont un faible coût et consomment moins d'énergie.
- Les signaux produits par un capteur sont traités par un microcontrôleur pour l'interprétation, l'analyse et la prise de décision.



10

Introduction à l'IoT

Actionneurs

Actionneur : C'est un élément mécanique ou électrique qui permet d'agir sur un système pour modifier son fonctionnement ou son état.

- Les actionneurs permettent de transformer l'énergie reçue en un phénomène physique (déplacement, dégagement de chaleur, émission de lumière …).
- Exemple : Haut-parleurs qui convertissent les signaux électriques correspondants en ondes sonores (acoustiques).



11

Introduction à l'IoT

Gateway dans l'IoT

- Une passerelle (gateway) est une combinaison de composants matériels et logiciels utilisés pour connecter un réseau à un autre.
- Les gateways permettent de relier les capteurs ou les nœuds de capteurs avec le monde extérieur.
- Les gateways sont donc utilisées pour la communication de données en collectant les mesures effectuées par les noeuds de capteurs et en les transmettant à l'infrastructure Internet.
- La gateway peut faire des traitements locaux sur les données avant de les relayer au Cloud.



12

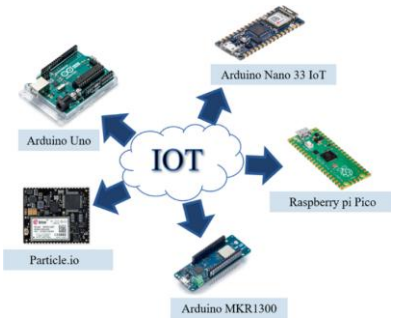
Introduction à l'IoT

Les cartes de développement pour l'IoT

L'important développement des plateformes « hardware » au cours de cette décennie a directement influencé l'évolution du domaine de l'IOT.

L'autonomie en énergie de ce type de système, ainsi que l'évolution des performances des processeurs, ont rendu l'intégration de plusieurs fonctions complexes plus simple et plus efficace.

Parmi ces plateformes, Arduino, Raspberry Pi, Particle, Intel Edison, STM, etc.



13

Introduction à l'IoT

Plateforme IoT

- Une plateforme IoT fournit **un ensemble de fonctionnalités prêtes** à l'emploi qui accélèrent considérablement le développement d'applications pour les appareils connectés et offrent une évolutivité et une compatibilité entre appareils.
- Les platesformes IoT ~ middleware IoT.
- Un middleware IoT fonctionne comme un médiateur entre les couches matérielles (hardware) et applicatives.
- Les tâches principales de ce middleware incluent:
  - ✓ la collecte de données à partir d'appareils sur différents protocoles et topologies de réseau
  - ✓ la configuration et le contrôle des appareils à distance
  - ✓ la gestion des appareils
  - ✓ les mises à jour des firmwares.

14

Introduction à l'IoT

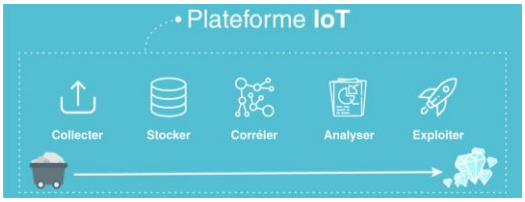
Exemples de plateformes IoT

Une plateforme d'IoT est un **ensemble de services** et chaque plateforme dispose d'avantages spécifiques en fonction de leurs services. Il existe trois types de plateformes :

Plateformes IoT propriétaires
- IBM Watson IoT Platform
- AmazonWeb Services (AWS) IoT Platform
- Microsoft Azure IoT Platform
- Google IoT Cloud Platform

Plateformes IoT Open-Source
- ThingsBoard
- myDevices (Cayenne)
- Blynk

Plateformes IoT basées sur l'industrie
- IoT platforms for vehicles
- IoT platforms for agriculture



15

Chapitre 2

La partie embarquée des objets connectés

16

La partie embarquée des objets connectés

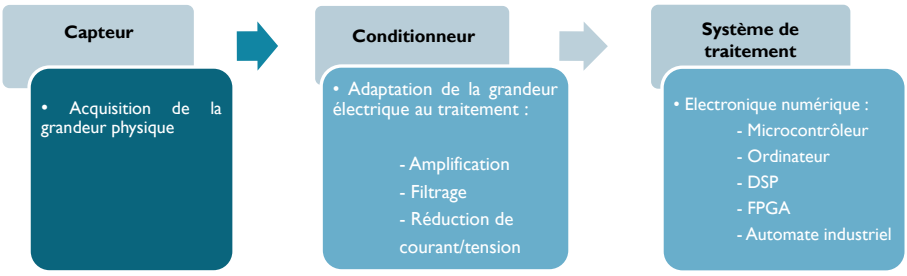
Partie hardware de l'objet connecté

- **Une plateforme matérielle d'acquisition et contrôle (MCU)**
  - Programmation matérielle (FPGA, ASIC, Automate, DSP …)
  - Programmation logicielle (µContrôleur, µprocesseur)
- **Des capteurs**
  - Analogique: Température, humidité, gaz, lumière…
  - Numérique: Présence, contact, impulsions…
- **Des actionneurs**
  - Relais, moteurs, lampes, variateurs…

17

La partie embarquée des objets connectés

Les capteurs



18

La partie embarquée des objets connectés

Les actionneurs

Système de commande

- Electronique numérique :
  - Microcontrôleur
  - Ordinateur
  - DSP
  - FPGA
  - Automate industriel

Conditionneur

- Adaptation de la grandeur électrique au traitement :
  - Isolation
  - Commutation
  - Conversion

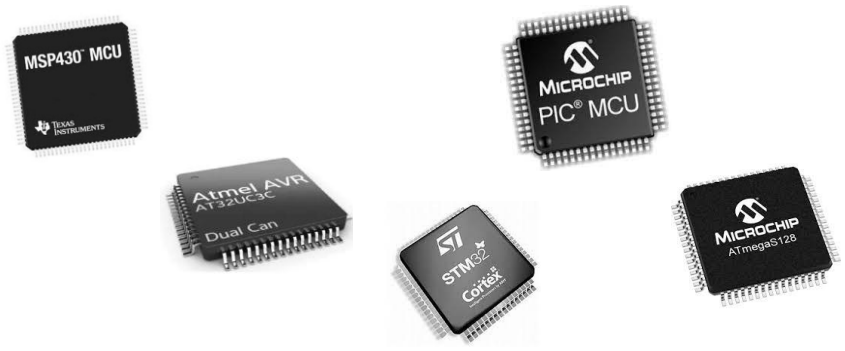
Actionneur

- Transformation de la grandeur électrique en action physique

19

La partie embarquée des objets connectés

Architectures à base de microcontrôleurs

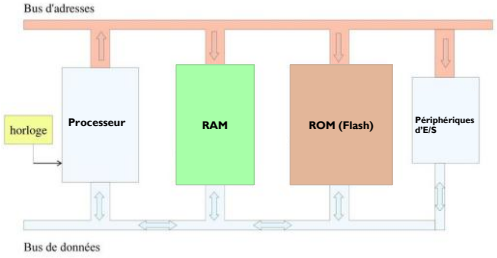


20

La partie embarquée des objets connectés

Les microcontrôleurs

- Processeur: élément central d'un système informatique
  - Interprète les instructions et traite les données.
- Besoins d'éléments complémentaires
  - Horloge pour le cadencer.
  - Mémoire pour l'exécution des programmes (RAM) et mémoire pour le stockage (ROM/Flash).
  - Périphériques d'entrées/sorties.
- Bus pour relier ces entités



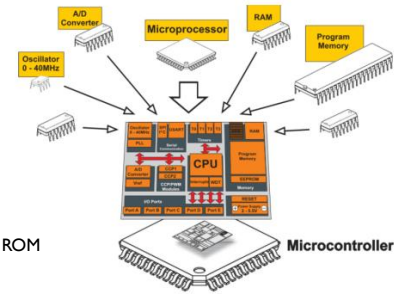
21

La partie embarquée des objets connectés

Microprocesseur et Microcontrôleur

Le microprocesseur
- Intégration dans des circuits distincts
- Nécessité de prévoir l'interconnexion (bus, câblage)
- La place occupée par ces composants séparés est plus importante.
- Plus de consommation et de chaleur dégagée

Le microcontrôleur
- Rassemble ces éléments sur un seul circuit intégré
- Composant autonome, capable d'exécuter des programmes sur sa ROM
- Moins de capacités que le microprocesseur

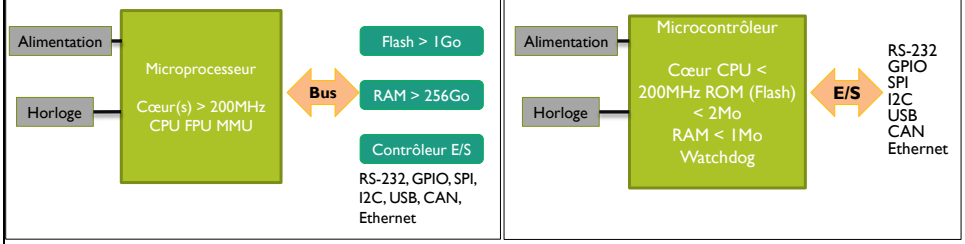


22

La partie embarquée des objets connectés

Microprocesseur vs Microcontrôleur

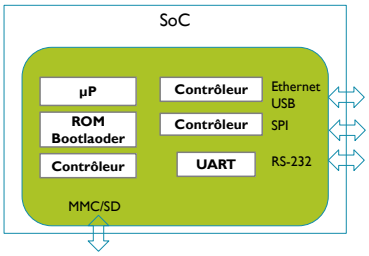
Microprocesseur	**Microcontrôleur**
• Electronique complexe • Entrées-sorties par des contrôleurs externes • Utilisation d'un OS	• Electronique simple • Fiabilité fonctionnement • Généralement sans OS



23

La partie embarquée des objets connectés

SoC : System on chip

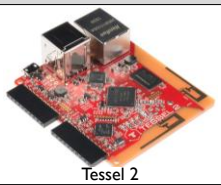


- Contrôleurs d'entrées-sorties déjà incorporés
- Intégration électronique plus complexe
- Souvent peu d'entrées sorties industrielles (CAN) ou analogiques (ADC/DAC, PWM)

24

La partie embarquée des objets connectés

Formes intégrées

Microcontrôleur		
 Arduino Uno	 MSP Launchpad	 STM 32 Discovery
SoC		
 Odroid	 Tessel 2	 Raspberry PI

25

La partie embarquée des objets connectés

Arduino

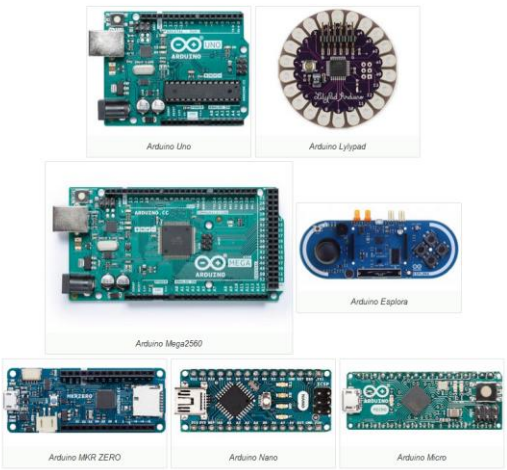
- Arduino est une plate-forme matérielle et logicielle électronique open source.
- Les cartes Arduino peuvent lire les entrées des capteurs (telles que la lumière, le son ou la pression) et les transformer en sorties (mouvement du moteur, allumer les LED et publier quelque chose en ligne).
- Arduino a été créé à l'Ivrea Interaction Design Institute pour permettre aux étudiants une formation en électronique et en programmation de faire du prototypage rapide



26

La partie embarquée des objets connectés

Arduino



27

La partie embarquée des objets connectés

Arduino shields

- Les Arduino shields sont des éléments qui peuvent être branchés sur une carte pour lui donner des fonctionnalités supplémentaires.
- Il existe de nombreux shields Arduino qui permettent aux cartes Arduino d'interagir avec le monde réel via différents capteurs physiques.
- En utilisant des shields Arduino, nous pouvons avoir une connectivité Arduino pour Internet pour utiliser les cartes Arduino dans l'IoT (la plupart des cartes Arduino n'ont pas de connectivité Internet par défaut).



28

La partie embarquée des objets connectés

Arduino avantages et inconvénients

Avantages:

- Simple à programmer et puissant.
- Peu coûteux
- Simple interface d'E/S
- Utiliser pour le prototypage rapide
- Logiciel open source et extensible
- Matériel extensible (Shields)
- Alimentation intégrée

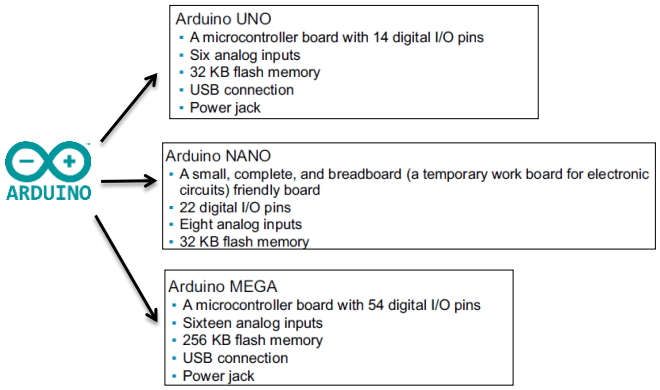
Inconvénients:

- Pas de connectivité sans fil
- Puissance de traitement limitée

29

La partie embarquée des objets connectés

Arduino caractéristiques techniques



30

La partie embarquée des objets connectés

Raspberry Pi

- Raspberry Pi est un nano-ordinateur pour soutenir l'apprentissage de la programmation.
- Plusieurs générations de Raspberry Pi ont été lancées. Tous les modèles sont équipés d'un système sur puce (SoC) avec une unité centrale de traitement (CPU) compatible ARM et une unité de traitement graphique (GPU) sur puce.
- Les cartes SD sont utilisées en général pour stocker le système d'exploitation et la mémoire du programme.
- Raspbian est le système d'exploitation officiel pour tous les modèles de Raspberry Pi. Il est basé sur Debian Linux et contient l'ensemble des programmes et utilitaires de base qui font fonctionner Raspberry Pi.

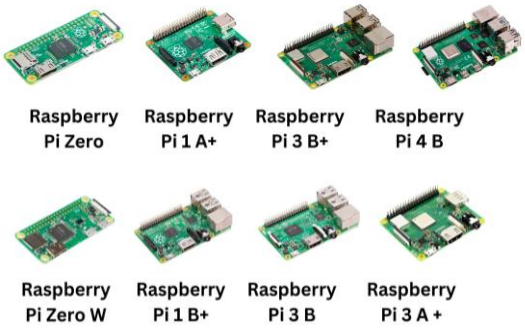


31

La partie embarquée des objets connectés

Raspberry Pi

- Plusieurs broches d'entrée/sortie à usage général (GPIO), qui prennent en charge les protocoles courants comme I²C.
- Les modèles B disposent d'un port Ethernet, et les Pi 3, Pi 4 et Pi Zero W intègrent le Wi-Fi 802.11n et le Bluetooth.



32

La partie embarquée des objets connectés

Raspberry Pi

Le Sense Hat est l'extension la plus populaire sur Raspberry Pi. C'est une carte créée par la Fondation Raspberry Pi, qui fournit de nombreux capteurs, une matrice LED et un joystick. Il est possible de programmer tout cela avec Python.

- Ce HAT dispose d'une matrice de 8X8 LED RGB, d'un joystick et de toutes une série de capteurs :

- Gyroscope
- Accéléromètre
- Magnétomètre
- Température
- Pression barométrique
- Humidité



33

La partie embarquée des objets connectés

Raspberry Pi avantages et inconvénients

Avantages:

- PC presque normal
- Puissance de traitement élevée
- Plusieurs interface d'E/S
- Langage de programmation multiple: Python, Java 7/8, C/C++, etc.
- Enorme répertoire SW
- Utiliser pour le prototypage
- Système d'exploitation complet

Inconvénients:

- Consommation d'énergie peu élevée
- Coût relativement élevé
- Pas de CAN (ADC)

34

La partie embarquée des objets connectés

Raspberry Pi Caractéristiques principales

Exemple: **Raspberry Pi 4**
•**Processeur** : Broadcom BCM2711, quad-core Cortex-A72 (ARM v8) 64-bit SoC @ 1.5GHz
•**RAM** : 8 Go LPDDR4
•**GPU** : VideoCore VI prenant en charge OpenGL ES 3.0, décodage HEVC 4K à 60 i/s
•**Connexion sans fil** : Bluetooth 5.0, Wi-Fi 802.11b/g/n/ac
•**Connexion filaire** : Gigabit Ethernet (RJ45)
•**Lecteur de carte micro-SD** (stockage non fourni)
•**Port caméra** CSI pour connecter la caméra Raspberry Pi
•**Port d'affichage** DSI pour connecter l'écran tactile Raspberry Pi
•**Audio** : AV 3.5 mm
•**Ports** : 2 x USB 3.0 / 2 x USB 2.0 / 1 x USB-C (alimentation seulement) / 1 x GPIO 40 pin / 1 x port quadripôle Audio/Vidéo composite / 2 x micro-HDMI
•**Alimentation :** 5V DC via un connecteur USB-C (minimum 3A), 5V DC via un en-tête GPIO (minimum 3A), compatible Power over Ethernet (PoE) (nécessite un HAT pour PoE)

Tous les modèles: https://www.creactivit.com/wp-content/uploads/2023/02/tableau-comparatif-raspberry-pi.pdf

35

La partie embarquée des objets connectés

Espressif

Les microcontrôleurs d'Espressif sont populaires parmi les développeurs IoT. L'entreprise est spécialisée dans la création de puces avec un haut niveau d'intégration ce qui aide à concevoir des appareils IoT faciles à fabriquer et à déployer.

Ces µC peuvent avoir différentes tailles mais la version la plus courante est l'ESP32

- L'ESP32 est une carte de développement à faible coût dédié à l'internet des objets (IoT) et les applications embarquées.
- C'est une petite carte électronique facile à prendre en main grâce à ses ressemblances avec la carte Arduino qui est bien plus répandue.



36

La partie embarquée des objets connectés

ESP32 avantages et inconvénients

Avantages:

- Faible coût
- Simple interface d'E/S
- Petite taille
- Possibilité d'utilisation dans l'environnement Arduino
- Utiliser pour le prototypage
- Basse consommation : l'ESP32 consomme très peu d'énergie par rapport aux autres microcontrôleurs, et il prend en charge les états de mode basse consommation comme le sommeil profond pour économiser de l'énergie ;
- Capacités Wi-Fi et Bluetooth
- Compatible avec MicroPython : peut etre programmer avec le micrologiciel MicroPython, qui est une ré implémentation de Python 3 ciblée pour les microcontrôleurs et les systèmes embarqués.

Inconvénients:

- Basse prise en charge des périphériques externes (capteurs, afficheurs, …)
- Alimentation non intégrée

37

La partie embarquée des objets connectés

ESP32 caractéristiques techniques

- **Processeur** : Xtensa **Dual-Core** 32-bit LX6 cadencé jusqu'à 240 MHz
- **Wi-Fi** (802.11b/g/n) et **Bluetooth** 4.2 / **BLE**
- **RAM** : **520 Ko** de SRAM interne et **4 Mo** de PSRAM externe
- **16 Mo** de Flash (externe)
- Différents états pour limiter la consommation d'énergie (Modem Sleep, Light Sleep, Deep Sleep, Hibernation)

38

La partie embarquée des objets connectés

Comparatif des cartes électroniques

	ESP32	ESP8266	Arduino UNO	Raspberry Pi Zero
Fréquence	160 Mhz	80 Mhz	16 Mhz	1 GHz
Ports Digitaux	36	17	14	28
Ports Analogiques	18	1	6	-
Bluetooth 4.0 Intégré	✓	-	-	✓
WiFi Intégré	✓	✓	-	✓
Alimentation USB	✓	-	✓	✓

39

La partie embarquée des objets connectés

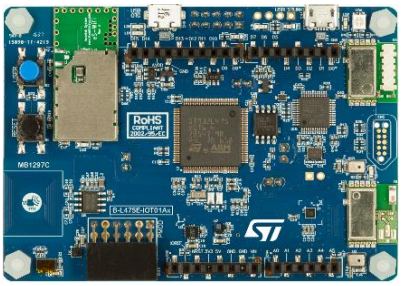
STM32L4 Discovery Kit IoT Node

- Connexion directe aux serveurs cloud (AWS)

- Communication longue portée à faible consommation

- Capteurs:
  - Humidité,
  - Pression,
  - Température
  - Mouvement,
  - Proximité,
  - Audio





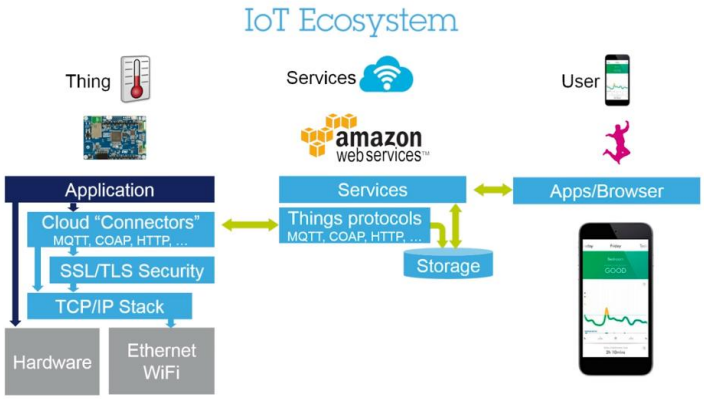
- PMOD : Interface de module périphérique prenant en charge GSM, GPS, etc…

- Connecteurs compatibles Arduino pour interfacer avec des cartes d'extensions supplémentaires.

40

La partie embarquée des objets connectés

STM32L4 Discovery Kit IoT Node



41

Chapitre 3

Plateformes IoT

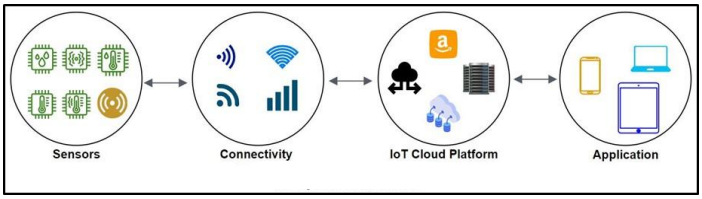
42

Plateformes IoT

Anatomie d'une solution IoT

Une plateforme IoT (appelée aussi middleware) fonctionne comme un médiateur entre les couches matérielles et applicatives.

Une plateforme IoT est un ensemble de services permettant de collecter, stocker, corréler, analyser et exploiter les données.



43

Plateformes IoT

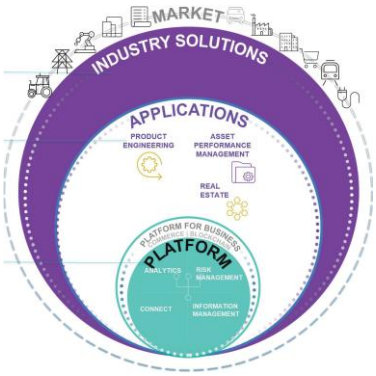
Solution d'une plateforme IoT

Une plateforme IoT importe les données des appareils et les transforme en informations significatives, qui peuvent optimiser les processus et guider la conception de nouveaux produits.

Les outils dont on a besoin pour créer, modifier, connecter, gérer, analyser et sécuriser les appareils et les données IoT.

Applications pour améliorer les résultats commerciaux grâce à des opérations et des produits connectés.

Des solutions pré-packagées pour développer le business des entreprises avec de nouveaux services et modèles commerciaux.



44

Plateformes IoT

Solution d'une plateforme IoT

Plateforme
- Facilite toutes les opérations de base pour les appareils en connexion.
- Gère et sécurise les données pour extraire des informations significatives.
- On peut déployer la plateforme avec activation des options de sécurité, ce qui peut être idéal pour une solution impliquant plusieurs parties qui doivent partager les données signalées à partir d'appareils IoT.

Applications
Grâce à la plateforme IoT, on peut activer les fonctionnalités de connectivité sur les applications existantes de la plateforme.

Solution industrielle
Il est possible de transformer le business des entreprises en utilisant des solutions pré-packagées qui utilisent les informations dérivées de la plateforme IoT.

45

Plateformes IoT

Platforme IoT Watson

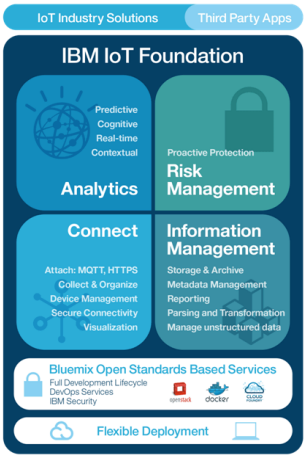
La plateforme IoT se compose de :

1. **Connexion:** connecter et gérer les appareils, les réseaux et les passerelles (gateway).

2. Gestion de l'information: intégration des données structurées et non structurées des appareils, des personnes et du monde qui nous entoure.

3. Services Analytiques : analyser et visualiser les données des appareils en temps réel à l'aide du Tableaux de bord de la plateforme.
- Obtenir des informations à partir des données en utilisant les données en temps réel, l'analyse prédictive et l'IA en exploitant les services de la plateforme.

4. Gestion des risques: exploiter les informations correctes provenant de sources fiables: la solution est visible de bout en bout.



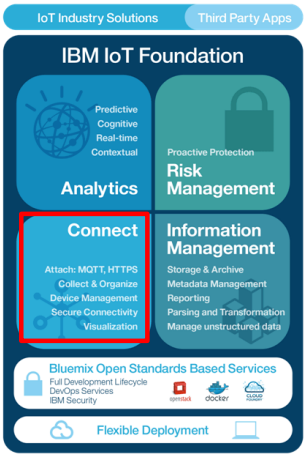
46

Plateformes IoT

Connexion des appareils à la plateforme IoT Watson

- Connecter et gérer les appareils, passerelles et réseaux IoT à l'aide d'une infrastructure large et croissante.
- Utiliser une communication basée sur des normes ouvertes (MQTT et HTTPS).
- Sécuriser la communication et la gestion.

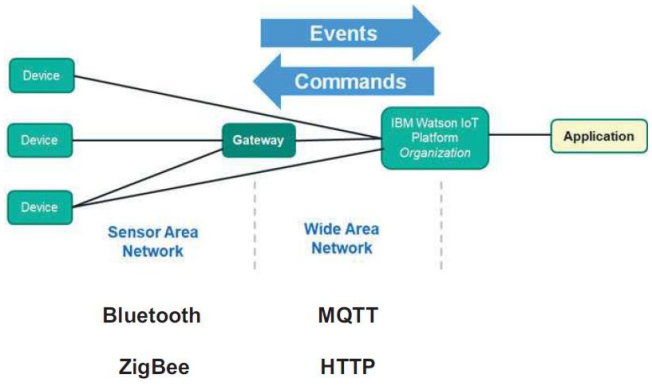
Connecter:
Importer des données IoT à partir de n'importe quelle source à l'aide de Watson IoT Platform Service pour enregistrer, connecter et contrôler les appareils IoT.
Activer la consommation sécurisée des données IoT et le contrôle des appareils à l'aide de Watson IoT Platform Registration Service.



47

Plateformes IoT

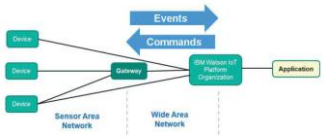
Connexion des appareils à la plateforme IoT Watson



48

Plateformes IoT

Connexion des appareils à la plateforme IoT Watson



Les appareils peuvent être connectés directement à la plateforme ou via une passerelle, selon le cas d'utilisation :

- Le **Sensor Area Network** est le protocole réseau utilisé pour communiquer les capteurs IoT à la passerelle. Les exemples incluent Bluetooth ou ZigBee.
- Les **événements** représentent les valeurs de données mesurées par le capteur. Par exemple, un événement peut être une lecture par le capteur de température d'une valeur >30 degrés. Dans de nombreux cas, les appareils n'ont pas à envoyer les données des capteurs au cloud en continu, mais uniquement lorsque cela est important, par exemple lorsqu'un événement se produit.
  Habituellement, les événements sont envoyés des appareils vers le cloud.
- Les **commandes** sont envoyées de Watson IoT Platform aux appareils. Les commandes peuvent être des commandes d'actionnement qui éteignent certains appareils ou des commandes de diagnostic afin que les appareils puissent signaler des dysfonctionnements.

49

Plateformes IoT

Connexion des appareils à la plateforme IoT

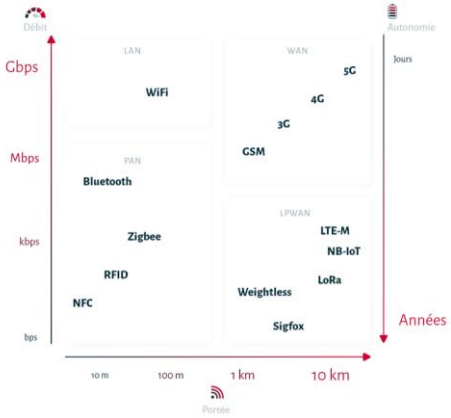
•**WAN** (Wide Area Network) : un réseau de plusieurs dizaines de kilomètres carrés

•**LPWAN** (Low Power Wide Area Network) : réseau de plusieurs dizaines de kilomètres carrés mais utilisant peu d'énergie (car peu de bande passante)

•**PAN** (Personal Area Network) : le réseau de quelques mètres (Bluetooth)

•**LAN** (Local Area Network) : le réseau Internet privé de domicile ou entreprise (Wifi)

•**Satellite** : partout dans le monde pour peu de ne pas être dans un tunnel (GPS)



50

Plateformes IoT

Protocoles de communication

- **MQTT (Message Queuing Telemetry Transport)** est un protocole de connectivité IoT de machine à machine (M2M) :
  - Il suit le protocole de publication et d'abonnement (publish and subscribe)
  - Il est basé sur le protocole TCP/IP.
- **HTTP (Hypertext Transfer Protocol)** est un protocole d'application utiliser pour activer la communication entre des systèmes physiquement dispersés. C'est l'un des protocoles les plus utilisés pour transférer des données sur Internet.

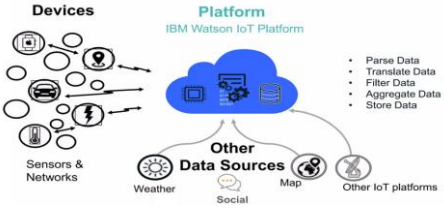
51

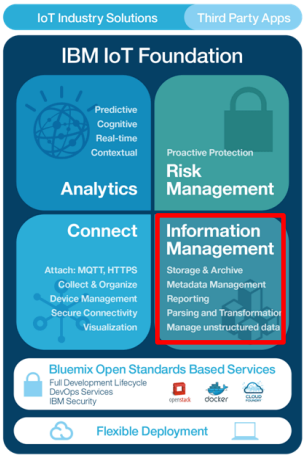
Plateformes IoT

Gestion de l'information

Exploiter et analyser les données de millions d'appareils en continu dans le cloud.

- Améliorer la visibilité des systèmes IoT grâce aux tableaux de bord et console.
- Intégrer les données critiques provenant d'autres plateformes telles que la météo et les émotions sociales.
- Tirer parti des fonctionnalités avancées de stockage, de mise en cache et de transformation des données.



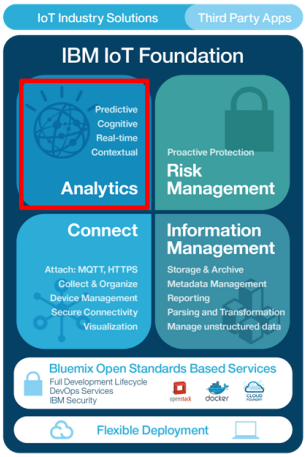


52

Plateformes IoT

Services Analytiques

- Surveiller les comportements des appareils en temps réel.
- Prendre les actions appropriées et prescrites automatiquement.
- Prévisions d'utilisation et de conditions d'exploitation en fonction des conditions environnementales.

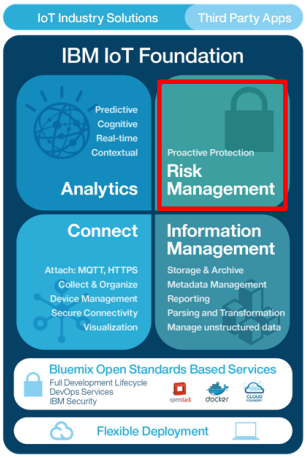


53

Plateformes IoT

Gestion des risques

- Appliquer le niveau approprié de sécurité et de confidentialité à la solution IoT, depuis l'appareil via le réseau jusqu'au cloud et au-delà.
- Fonctionnalités de sécurité de base de bout en bout pour les appareils, les données et les connexions.
- Gagner en confiance du système
- Fournir un support pour les systèmes décentralisés sécurisés tels que Blockchain



54

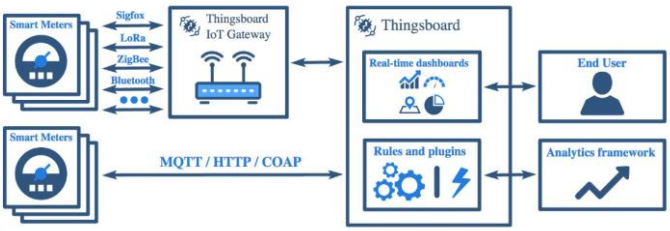
Plateformes IoT

ThingsBoard

Thingsboard est une plateforme IoT open source qui permet un développement, une gestion et une mise à l'échelle rapides des projets IoT.

ThingsBoard assure la collecte, le traitement et la visualisation des données ainsi que la gestion des appareils.

- Elle permet la connectivité des périphériques via les protocoles IoT standard tel que : MQTT, CoAP et HTTP.
- Elle prend en charge les déploiements en local et sur le cloud.



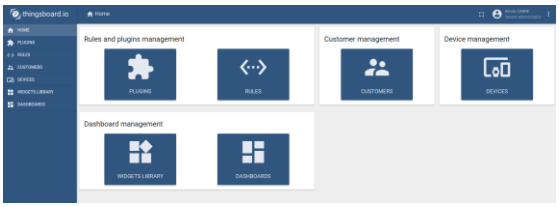
55

Plateformes IoT

ThingsBoard principales caractéristiques et fonctionnalités

Gestion de l'approvisionnement, la surveillance et le contrôle des objets connectés de manière sécurisée à l'aide d'API (Application Programming Interface) côté serveur. Définition des relations entre les appareils, les clients ou toute autre entité.

Collection et visualisation des données des périphériques de manière évolutive. Grâce aux widgets intégrés ou personnalisés et des tableaux de bord flexibles. Ainsi, le partage des tableaux de bord avec les clients de la plateforme.



56

Plateformes IoT

ThingsBoard principales caractéristiques et fonctionnalités

Le moteur de règles (Rule Engine) permet de :

- Définir les chaînes de règles de traitement de données.
- Génération des alarmes sur les événements de télémétrie entrants,
- Mise à jour des attributs en cas de l'inactivité des périphériques.
- Transformer et normaliser les données des appareils.

Gestion des alarmes liées aux appareils, clients actifs...etc. La surveillance des alarmes en temps réel et la propagation d'alarmes vers la hiérarchie des entités associées. Propagation des alarmes lors d'événements de déconnexion ou d'inactivité de l'appareil.

Offrir une API REST, une interface utilisateur qui prend en charge diverses entités qui travaillent ensemble pour atteindre un objectif particulier consistant à fournir et à gérer plusieurs types d'entités et leurs relations dans l'application IoT.

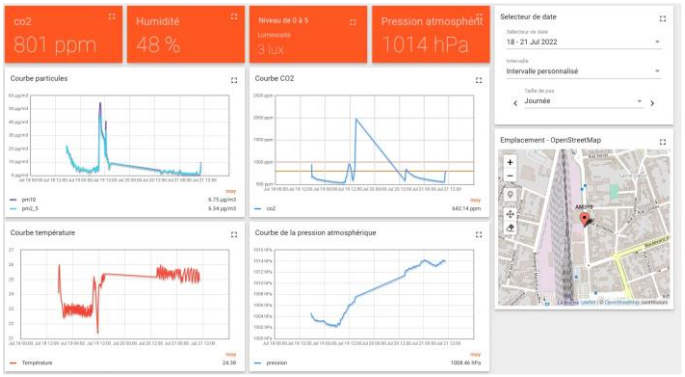
L'évolutivité horizontale est prise en compte si le nombre de requêtes côté serveur augmente de manière linéaire. De nouveaux serveurs Thingsboard peuvent être ajoutés en mode cluster. Ceci est bénéfique pour les applications critiques sans temps d'arrêt.

57

Plateformes IoT

Exemple

Un exemple de données remontées par des capteurs Lorawan (capteur de CO2, de pression atmosphérique, de températures, etc.):



Chapitre 4

Cloud Computing

Cloud Computing

Cloud

Un **réseau de ressources informatique** comme : réseaux, serveurs, espace du stockage, applications et services , accessible publiquement ou en privé sur internet.



Quel intérêt ?

L'utilisation du Cloud permet également d'assurer **l'évolutivité**.

Lorsque on dispose de centaines, de milliers, voire de millions de capteurs, il serait extrêmement coûteux et énergivore d'affecter de grandes quantités de puissance de calcul à chaque capteur.

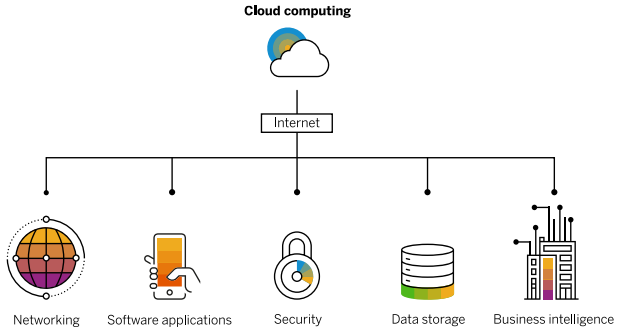
➔ Au lieu de cela, les données de tous ces capteurs peuvent être transmises au Cloud et y être traitées de manière agrégée.

Cloud Computing

Définition



Le **Cloud Computing** (informatique dans le nuage) est une technique permettant de gérer des ressources informatiques partagées et configurables par exemple, réseaux, serveurs, stockage, applications et services qui peuvent être rapidement mise à disposition des utilisateurs.

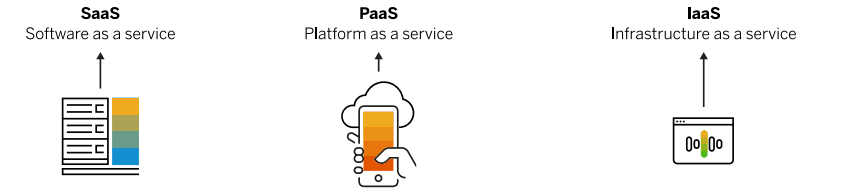


61

Cloud Computing

Services de Cloud Computing

Le Cloud Computing se divise en trois grandes catégories de services : SaaS, PaaS et IaaS. Certains prestataires combinent ces services et d'autres les proposent indépendamment les uns des autres.



62

Cloud Computing

SaaS: Software as a Service (Logiciel en tant que service)

Logiciel consommé en tant que service:

- Le fournisseur de SaaS Cloud possède et gère l'ensemble de sa plateforme (du matériel au logiciel).
- Dans ce modèle, le client cloud utilise le logiciel mais ne s'occupe pas de la pile ci-dessous (plateforme d'application, matériel ...) ou de l'installation du logiciel
- Dans une solution SaaS, le contrôle des données est partagé entre le client (qui crée et utilise les données) et le fournisseur de cloud (qui héberge, stocke, sécurise et sauvegarde les données).



Il existe de nombreux exemples populaires de SaaS, notamment Google Apps, Office 365, Citrix, Salesforce, Netflix, Cisco WebEx, etc.

63

Cloud Computing

Paas : Platform as a Service (Plateforme en tant que service)

- Une plateforme sur laquelle les développeurs ou les éditeurs de logiciels peuvent déployer des applications.
- Environnements de développement prêts à l'emploie et leur permet de créer, tester, déployer et gérer des applications diverses.
- La pile (la plateforme d'application, le système d'exploitation, le matériel, le réseau) est détenue et gérée par le fournisseur de services.



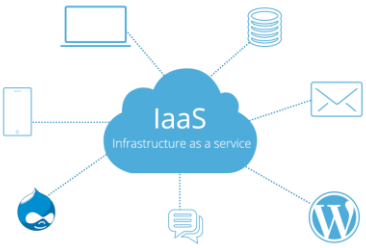
Parmi les fournisseurs PaaS existants, Microsoft Azur , IBM Cloud, AWS ou encore ThingsBoard.

64

Cloud Computing

IaaS : Infrastructure as a Service (Infrastructure en tant que service)

- L'utilisateur gère les ressources de calculs et de stockage.
- La gestion de l'infrastructure physique et virtuelle c'est la responsabilité du fournisseur.
- L'utilisateur a le contrôle du système d'exploitation, de stockage, et des applications déployées.



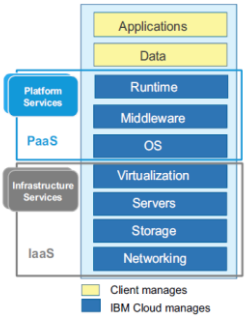
Parmi les fournisseurs IaaS existants, Amazon EC2, Microsoft Azur, IBM Cloud ou encore Google Cloud.

65

Cloud Computing

IBM Cloud

- Une plateforme de cloud computing ouverte et basée sur des normes.
- Comprend un catalogue de divers services de plateforme et d'infrastructure.
- Combine une plateforme en tant que service (PaaS) avec une infrastructure en tant que service (IaaS).
- Utilisé pour créer, déployer et exécuter rapidement des applications d'entreprise , infrastructures ou les deux.

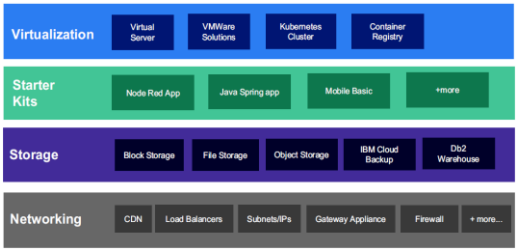


66

Cloud Computing

IaaS de IBM Cloud

- IaaS permet de déployer une infrastructure hautes performances dans les centres de données IBM Cloud dans le monde entier : serveurs bare metal, serveurs virtuels, conteneurs, stockage et mise en réseau.
- IaaS fournit des services pour déployer, accéder et gérer l'infrastructure.

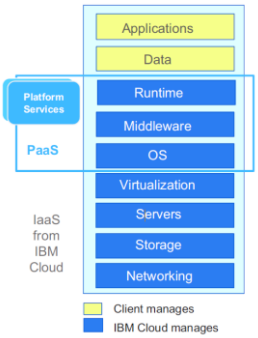


67

Cloud Computing

PaaS de IBM Cloud

- PaaS permet de créer, gérer et exécuter des applications.
- PaaS utilise **Cloud Foundry**, une offre de PaaS open source qui fournit un choix de clouds, de frameworks et de services d'application:
  - ✓ Étendre Cloud Foundry avec les services d'IBM et des partenaires commerciaux d'IBM.
  - ✓ Fournir un scriptable command-line interface (CLI).
  - ✓ S'intègrer au développement des outils, tels qu'Eclipse, pour faciliter le processus de déploiement.
- PaaS peut s'exécuter dans les centres de données IBM Cloud dans le monde entier.



68

Cloud Computing

Cloud Foundary

CLOUD FOUNDRY

- Cloud Foundry est le premier standard de l'industrie PaaS qui garantit le déploiement le plus rapide, le plus simple et le plus fiable des applications natives Cloud.
- Cloud Foundry garantit que les aspects de développement et de déploiement du codage restent soigneusement coordonnés avec tous les services associés pour garantir des itérations rapides, cohérente et fiable pour les applications.
- Cloud Foundry possède une architecture basée sur des conteneurs qui exécute des applications avec n'importe quel language de programmation. (Vous pouvez déployer des applications sur Cloud Foundry en utilisant vos outils existants sans aucune modification du code.)

cf push — And Your App is Live



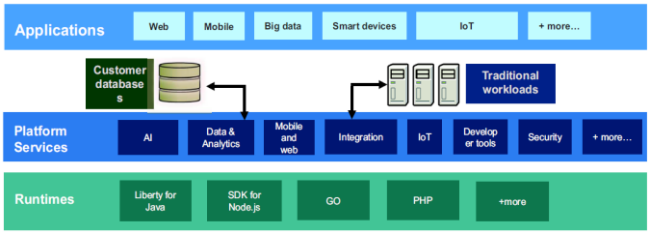
69

Cloud Computing

PaaS de IBM Cloud

Les ressources suivantes sont fournies sur IBM Cloud :

- Runtimes sur lesquels exécuter des applications.
- Catalogues de services pouvant être utilisés pour créer des applications:
  - ❑ Databases
  - ❑ Mobile and web support
  - ❑ Data & Analytics
  - ❑ Artificial intelligence (AI)
  - ❑ Security
- Intégration avec les données et les charges de travail traditionnelles tournant sur des systèmes sur site.
- Capacités et outils DevOps: code editor, version control,…

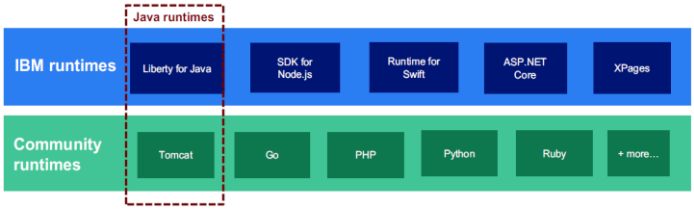


70

Cloud Computing

IBM Cloud: Choix du runtime

- Avec IBM Cloud, on peut exécuter une application à l'aide d'un environnement d'exécution sans avoir à gérer l'infrastructure.
- Un runtime est un ensemble de ressources utilisées pour exécuter une application:
  - Pour chaque runtime, un IBM community buildpack fournit les scripts pour préparer le code à s'exécuter sur IBM Cloud.
  - IBM Cloud et Cloud Foundry prennent en charge encore plus d'environnements d'exécution via les community Buildpacks.

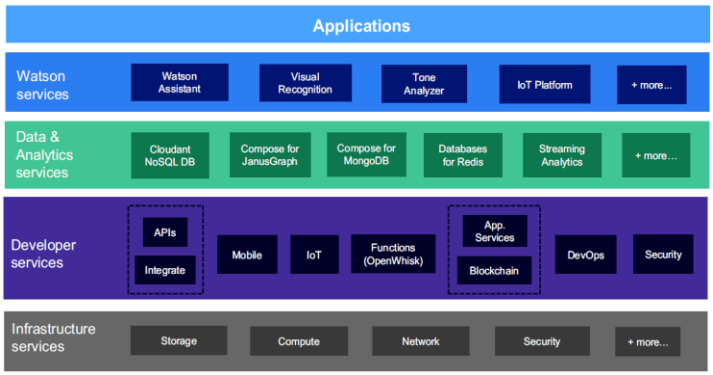


71

Cloud Computing

IBM Cloud: Services

Les services prédéfinis fournissent des blocs de construction pour les applications riches en fonctionnalités.



72

Cloud Computing

IBM Cloud: Services

- Avec les services Watson, on peut ajouter la puissance de l'IA à l'application avec des API de parole, de vision et de traitement du language naturel (NLP).



- Les services de données et d'analyse aident à obtenir des données à partir de bases de données cloud intégrées, à créer des applications basées sur les données et à analyser les données.



- Les services d'infrastructure aident à gérer l'infrastructure sous-jacente sur laquelle l'application s'exécute.



73

Cloud Computing

IBM Cloud: Services

De plus, les services de catégories « Develpoper services » aident à développer des fonctionnalités clés au sein l'application :

- Services d'intégration :
  - API : créer, gérer, appliquer et exécuter des API.
  - Integrate : accéder aux charges de travail traditionnelles qui s'exécutent dans l'environnement sur site de l'organisation.
- Mobile : utiliser une infrastructure backend mobile pour créer, surveiller et tester des applications mobiles.
- IoT : communiquer avec les appareils, les capteurs et les passerelles connectées.
- Fonctions : exécuter en réponse aux événements entrants (basé sur Apache OpenWhisk).
- Application Services : de nombreux services d'application, tels que la blockchain, Message Hub, WebSphere Application Server, Business Rules et d'autres services d'application, se trouvent sur le cloud.
- DevOps : outils pour aider à innover pour la création de nouvelles applications plus rapidement et moins cher.
- Sécurité : intégrer la sécurité dans la conception de l'application.



74

Cloud Computing

IBM Cloud: régions et emplacements

- IBM Cloud est hébergé dans le monde entier.
- Une région est une zone géographique où l'application est hébergée
- Les emplacements sont des centres de données dans une région où le matériel est utilisé.

- Il faut sélectionner une région pour déployer l'application :
  - ✓ La région la plus proche des utilisateurs peut fournir de meilleures performances (latence).
  - ✓ Une région spécifique peut répondre aux exigences de sécurité des données.
  - ✓ Plusieurs régions offrent une haute disponibilité.

IBM Cloud Kubernetes Service region	Corresponding IBM Cloud location	Single zone Location
AP North (standard clusters only)	Tokyo	Singapore, Chennai, Seoul, and Hong Kong S.A.R. of the PRC
AP South	Sydney	Melbourne
EU Central	Frankfurt	Amsterdam, Milan, Oslo, and Paris
UK South	London	
US East (standard clusters only)	Washington DC	Montreal and Toronto
US South	Dallas	San Jose and Sao Paulo

75

Cloud Computing

Databases sous IBM Cloud

IBM Cloud fournit les principaux types de bases de données et propose des services pour chaque type :

- **Les bases de données NoSQL** ont l'avantage de stocker les objets dans le même format attendu par les applications. Les données sont transférées et stockées dans des formats standards, tels que JSON ou XML.
- **Un service de base de données SQL** fournit des bases de données basées sur des tables relationnelles. Ils ont été largement utilisés pour stocker des données dans l'informatique d'entreprise (enterprise computing).
- Une base de données en colonnes **« Columnar »** est la base de données qui stocke les données sous forme de colonnes plutôt que de lignes .
- Les services de données de paires clé-valeur (**Key-value pair data**) permettent un stockage efficace des données. Une paire clé-valeur se compose de deux éléments de données associés : Une clé, qui est une constante qui définit le jeu de données et une valeur, qui est une variable qui appartient à l'ensemble.

76

Cloud Computing

Amazon Web Services (AWS)

- AWS est une plateforme cloud sécurisée qui offre un large éventail de produits basés sur le cloud.
- AWS offre plus de 165 services différents qui incluent des solutions.
- AWS offre un accès à la demande au calcul, au stockage, réseau, base de données et autres ressources informatiques et outils de gestion.
- AWS fournit à la fois des services de PaaS et de IaaS, mais aussi quelques services en SaaS.



77

Cloud Computing

IaaS de AWS

- IaaS est l'un des services Web d'Amazon qui se concentre sur la fourniture de services d'infrastructure basés sur la technologie du cloud computing.
- IaaS Amazon Service est utilisé pour remplacer les ressources physiques, telles que les serveurs, par des ressources virtuelles hébergées et gérées par Amazon.
- Les utilisateurs du système peuvent exécuter n'importe quel système d'exploitation ou application sur ces serveurs loués, sans encourir de frais supplémentaires pour la maintenance et l'exploitation.



78

Cloud Computing

PaaS de AWS

- AWS PaaS fournit aux développeurs une base pour concevoir des applications.
- Parmi 3 solutions de services Amazon qui appliquent la technologie du cloud computing, à savoir Iaas Paas Saas AWS, PaaS joue un rôle important dans la simplification du processus de développement d'applications.
- Grâce à la technologie cloud, les développeurs peuvent accéder aux données de la plateforme depuis n'importe où. Cela peut faciliter le développement des applications. Cependant, cela signifie également que les développeurs auront moins de contrôle sur l'environnement de conception des applications.
- Example : AWS IoT.



79

Cloud Computing

SaaS de AWS

- Amazon inclut également le SaaS dans ses services.
- Le SaaS est également l'un des services Web d'Amazon privilégiés par un grand nombre d'utilisateurs dans le monde en raison des multiples avantages qu'il apporte.



SaaS ou PaaS ou IaaS

- Quant à l'offre de fonctionnalités, les trois Amazon Web Services IaaS, PaaS et SaaS se distinguent les uns des autres par lesquels :
- IaaS offre la plus grande flexibilité pour l'hébergement d'applications personnalisées, ainsi qu'un centre de données général pour le stockage des données.
- Le PaaS permet d'investir plus de temps dans le développement de l'application tout en ayant l'infrastructure gérée par le fournisseur.
- Le SaaS fournit des solutions prêtes à l'emploi pour répondre à un besoin métier spécifique.

80

Cloud Computing

Catégories de services AWS

AWS propose de nombreux services. Le choix de service dépend des objectifs commerciaux et des exigences technologiques.



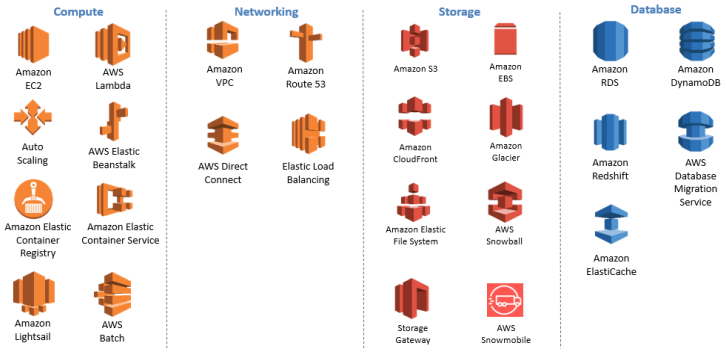


81

Cloud Computing

Les services de base de AWS

- Les services de base font référence à un groupe large et approfondi de services d'infrastructure cloud de base.
- Les services AWS sont généralement regroupés en différentes catégories telles que le calcul, la mise en réseau, le stockage, les applications, les bases de données et les analyses.

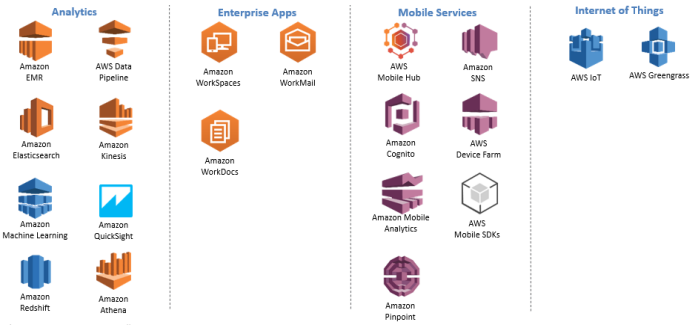


82

Cloud Computing

Les services fondamentaux de AWS

- Les services fondamentaux font référence à des groupes de services qui offrent des solutions basées sur le cloud pour les plateformes d'analyse, d'entreprise, mobiles et Internet des objets (IoT).



83

Cloud Computing

Les outils de développement AWS

- Les outils de développement AWS sont un ensemble de services conçus pour permettre aux développeurs et aux professionnels des opérations informatiques pratiquant DevOps de fournir rapidement et en toute sécurité des logiciels via des outils de gestion, des services de sécurité et d'identification et des applications.



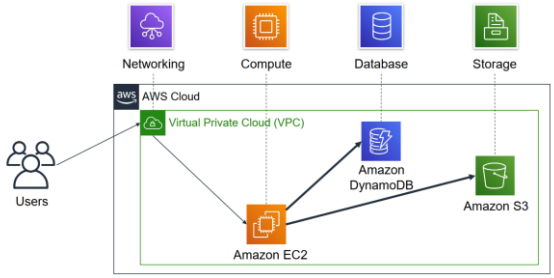
84

Cloud Computing

Exemple de solution simple

Au départ, pour commencer à utiliser le cloud, il faut se concentrer sur quelques services « de base ». Plus précisément, les services de:

- Calcul : y compris Amazon Elastic Compute Cloud (ou Amazon EC2), AWS Lambda et AWS Elastic Beanstalk.
- Mise en réseau : y compris Amazon Virtual Private Cloud (ou VPC), Amazon Route 53 et Domain Name Services.
- Stockage : y compris Amazon S3 (Simple Storage Service) et Amazon Glacier.
- Bases de données notamment Amazon RDS (Relational Database Service) et Amazon DynamoDB (Non-Relational Database).



85

Cloud Computing

EC2: Elastic Compute Cloud

Ce service permet de gérer des serveurs sous forme de machines virtuelles dans le cloud (vous pouvez lancer des serveurs et faire ce que vous voulez avec. Vous avez accès à la ligne de commande, donc vous pouvez les piloter à distance).
Il compose l'offre de IaaS.

Amazon **EC2**

RDS : Relational Database Service

Ce service permet de gérer des bases de données managées dans le cloud (un serveur de base de données préconfiguré).
L'utilisateur se concentre sur l'utilisation de la base de données et Amazon s'occuper de toute la gestion du serveur (ex. : mises à jour de sécurité et sauvegarde).

S3 : Simple Storage Service

Amazon S3 (Simple Storage Service) est un service de stockage et de distribution de fichiers.
C'est une sorte d'entrepôt de fichiers à très bas coût qui garantit de ne jamais perdre vos données.
(Utilisez-le pour faire télécharger des fichiers sur votre site, ou pour y stocker des images).

Amazon **S3**

86

Cloud Computing

Interagir avec AWS



- **Console de gestion AWS** : Interface graphique facile à utiliser



- **Interface de ligne de commande (AWS CLI)** : Accès aux services par commandes discrètes ou scripts



- **Kits de développement logiciel (SDK)** : Accédez aux services directement à partir de votre code (tels que Java, Python et autres)

87

Chapitre 5

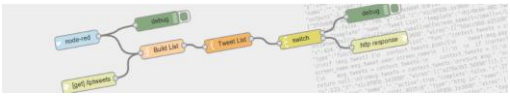
Node-RED

88

Node-RED

Présentation

- Node-RED a été créé par IBM en 2013 afin de faciliter la création d'objets connectés. Il fait désormais partie de la fondation JS depuis 2016 et est diffusé en open-source.
- **Node-RED** est un outil puissant pour construire des applications IoT en mettant l'accent sur la simplification de la programmation qui se fait grâce à des blocs de code prédéfinis, appelés « **nodes** » pour effectuer des tâches.



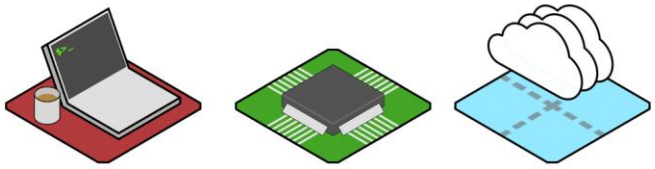
- Node-RED est un outil de programmation graphique permettant de connecter des périphériques matériels, des API et des services en ligne pour faciliter l'intégration des IoT .

89

Node-RED

Plateformes

Node-RED peut être déployé sur des environnements locaux (ordinateur), sur des appareils principalement en Edge sur un système embarqué (android, raspberry), et sur le cloud (IBM, AWS).



Run locally
- Getting started
- Docker

On a device
- Raspberry Pi
- BeagleBone Black
- Interacting with Arduino
- Android

In the cloud
- IBM Cloud
- SenseTecnic FRED
- Amazon Web Services
- Microsoft Azure

90

Node-RED

Quel intérêt ?

1- Développement rapide pour les applications IoT

- Facilite la création d'applications IoT en quelques heures sans codage.
- Mêmes styles de développement dans les deux environnements Edge et Cloud.

2- Technologies standards en IoT industriel.

- Node-Red prend en charge les technologies IoT essentielles par exemple REST API ,WebSocket et MQTT.
- Logiciel préinstallé sur les appareils Edge standard.



91

Node-RED

Que peut-on faire avec Node-RED?

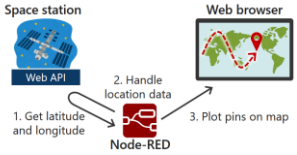
Node-RED a des possibilités infinies.Voici quelques exemples de ce que les utilisateurs ont réalisé avec :

- Passerelle de capteurs IoT (en utilisant MQTT)
- Création de services API
- Automatisation de courriels (e-mails)
- Passerelle entre des services tels que des API et des bases de données







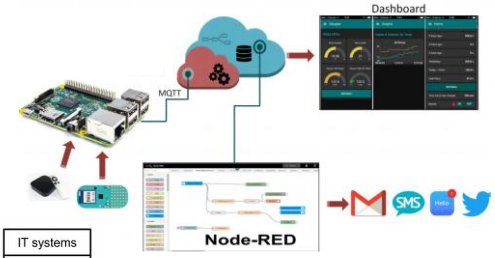


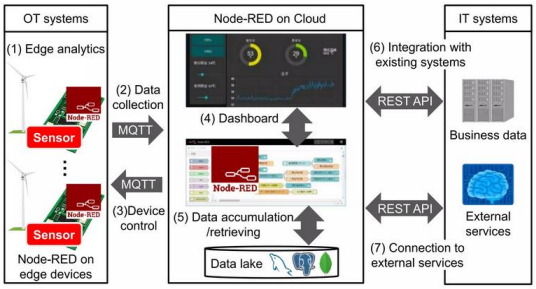
92

Node-RED

Architecture IoT basé sur Node-RED

Node-RED possède des fonctionnalités clés pour réaliser une application IoT.



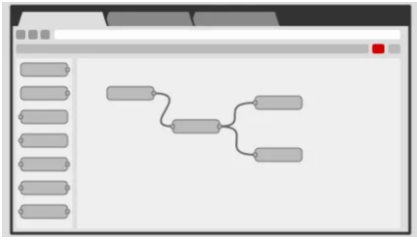


93

Node-RED

Caractéristiques du Node-RED

- Edition de flux par navigateur:
  - Node-RED fournit un éditeur de flux basé sur un navigateur qui facilite l'assemblage des flux en utilisant la large gamme de nœuds de la palette. Les flux peuvent ensuite être déployés vers le runtime en un seul clic.
  - Les fonctions JS peuvent être créées dans l'éditeur à l'aide d'un éditeur de texte riche.
  - Une bibliothèque intégrée enregistre des fonctions, des modèles ou des flux utiles pour les réutiliser.

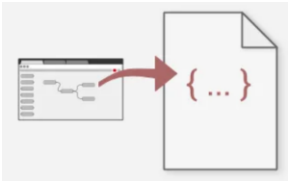


94

Node-RED

Caractéristiques du Node-RED

- Construit sur Node.js :
  - Possède tous les avantages de son modèle événementiel et non bloquant.
  - Il est facile d'étendre la gamme de nœuds de palette pour ajouter de nouvelles fonctionnalités.

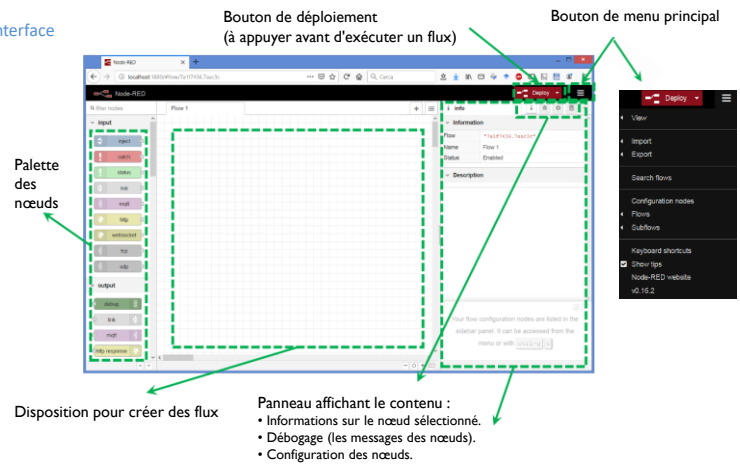


- Développement social:
  - Les flux créés dans Node-RED sont stockés à l'aide de fichiers JSON, qui peuvent être facilement importés et exportés pour être partagés avec d'autres.
  - Une bibliothèque de flux en ligne qui permet le partage des flux avec le monde entier.

95

Node-RED

Node-RED interface

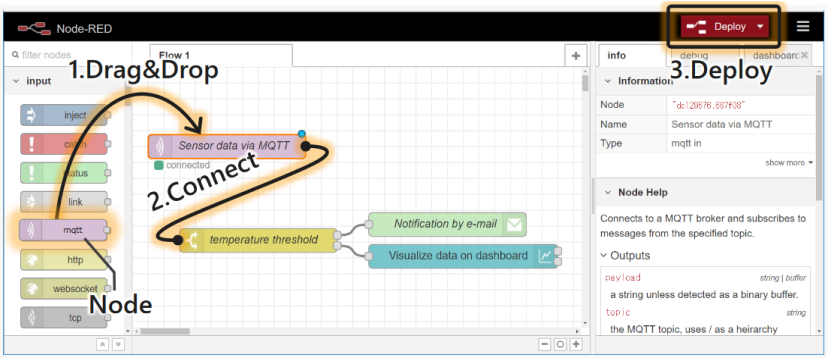


96

Node-RED

Prototypage rapide

Développement facile et rapide en connectant des nœuds.
Plus de 1400 nœuds disponibles, y compris la technologie IoT essentielle.



97

Node-RED

Composants de flux

- Node-RED offre de puissants éléments de base qui permet de créer rapidement des flux.
- Les nœuds connectés, généralement une combinaison de nœuds d'entrée, de nœuds de traitement et de nœuds de sortie, lorsqu'ils sont câblés ensemble, constituent un flux.

Les trois types de base

Autres ensembles qui sont une combinaison de nœuds d'entrée, de sortie et de fonction



98

Node-RED

Nœuds d'entrée

- Les nœuds d'entrée sont des nœuds qui sont utilisés pour démarrer le flux.
- Il y a sept nœuds d'entrée de base qui sont installés par défaut.
- Au moins un nœud d'entrée doit être inclus dans le flux de messages.

Exemples:

Inject Node: injecte un horodatage (timestamp) ou un texte configuré par l'utilisateur dans un message. Il peut être configuré pour injecter manuellement, à un intervalle défini ou à des heures spécifiques.





Mqtt node: s'abonne à un MQTT broker et écoute un sujet (topic). Il renvoie toutes les données publiées sur le sujet en tant que nouveau message.





99

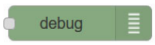
Node-RED

Nœuds de sortie

- Les nœuds de sortie sont essentiellement les images miroir de l'ensemble des nœuds d'entrée et fournissent un moyen d'envoyer des données sur le même ensemble de protocoles.
- Les nœuds de sortie sont les points de terminaison du flux. Ils sont utilisés pour envoyer des données à un service externe ou à un pin d'un composant, et ils peuvent générer des messages de réponse.

Exemple:

Debug node : fournit un moyen simple d'afficher les messages, qui seront affichés dans le volet de débogage.





100

Node-RED

Nœuds de fonctions

- Le type « function node » contient divers nœuds qui exécutent des fonctions de traitement spécifiques.
- Les fonctions varient de simples nœuds de délai « delay » et de commutation au nœud de fonction programmable qui peut être adapté à presque tous les besoins de programmation.
- Ils sont utilisés lorsqu'il n'y a pas de nœud existant dédié à votre tâche.

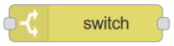
Exemple:

Function node : permet au code JS d'être exécuté sur les messages qui lui sont transmis.





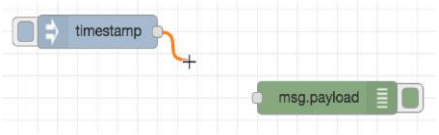
Switch node : achemine les messages en fonction de leurs propriétés.



101

Node-RED

Fils (wires)



- Ce sont les fils de connexion qui définissent les connexions entre les points de terminaison d'entrée et de sortie des nœuds dans un flux.
- Les messages ont généralement une propriété de charge utile (payload), qui est la propriété par défaut avec laquelle la plupart des nœuds fonctionnent pour afficher les données.

N.B: Node-RED ajoute également une propriété appelée _msgid, qui est un identifiant pour le message qui peut être utilisé pour suivre sa progression à travers un flux.

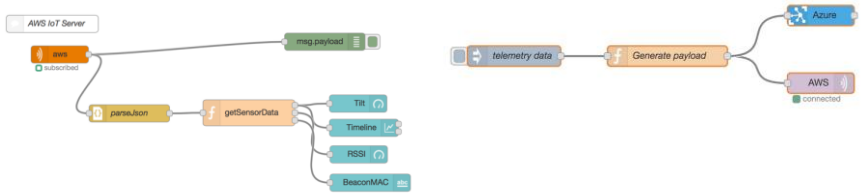


102

Node-RED

Node-RED et Cloud

- La plupart des Cloud fournit des nœuds Node-RED pour vous aider à connecter vos appareils, passerelles et applications à une Platform IoT et à créer rapidement des solutions IoT.
- Une intégration simple, rapide et standard.

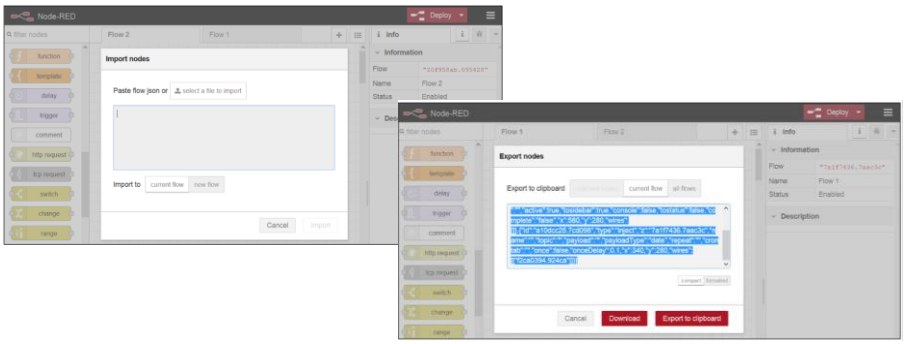


103

Node-RED

Flux Import et Export

Les flux peuvent être importés et exportés depuis l'éditeur en utilisant le format JSON.

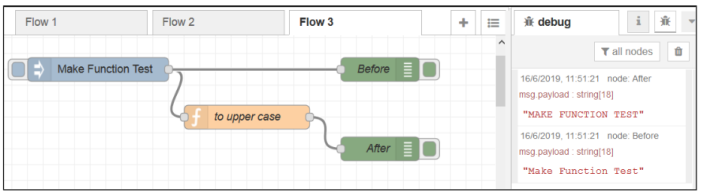


104

Node-RED

Flux Import et Export

Exemple



Function body:

```
msg.payload = msg.payload.toUpperCase()
return msg;
```


```
[{"id":"c6a5d62f.46d2f","type":"tab","label":"Flow 3","disabled":false,"info":""},{"id":"ec2f106.b8260f","type":"function","z":"c6a5d62f.46d2f","name":"to upper case","func":"msg.payload = msg.payload.toUpperCase()\nreturn msg;","outputs":1,"noerr":0,"x":320,"y":180,"wires":[["c011ed5d.75eda"]]},{"id":"cdefe980.e69b4","type":"inject","z":"c6a5d62f.46d2f","name":"","topic":"","payload":"Make Function Test","payloadType":"str","repeat":"","crontab":"","once":false,"onceDelay":0.1,"x":130,"y":120,"wires":[["ec2f106.b8260f","91e5aad8.61ea78"]]},{"id":"c011ed5d.75eda","type":"debug","z":"c6a5d62f.46d2f","name":"After","active":true,"tosidebar":true,"console":false,"tostatus":false,"complete":"payload","targetType":"msg","x":490,"y":220,"wires":[]},{"id":"91e5aad8.61ea78","type":"debug","z":"c6a5d62f.46d2f","name":"Before","active":true,"tosidebar":true,"console":false,"tostatus":false,"complete":"payload","targetType":"msg","x":490,"y":120,"wires":[]}]
```

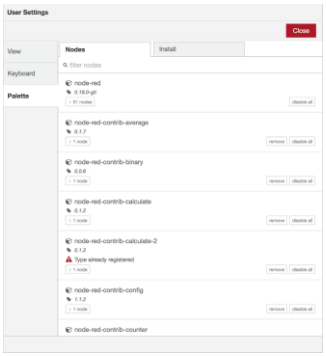

105

Node-RED

Gestionnaire de palettes

- Comme tous les nœuds ne se trouvent pas dans la palette, vous pouvez utiliser le gestionnaire de palettes pour installer de nouveaux nœuds.

- Le gestionnaire de palettes comporte deux onglets :
  - Nœuds
  - Install

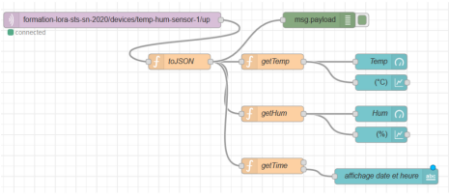


106

Node-RED

Node-RED Dashboard

- Dans la grande majorité des applications industrielles, le temps réel est obligatoire pour contrôler tout processus et actionnement.
- Le nœud Dashboard vous permettra de créer votre propre écran pour surveiller et contrôler, via des graphiques et des boutons, vos appareils, machines et capteurs.
- Configurez votre propre affichage de données et y accédez via le Web.



Résultat



107

Node-RED

Node-RED Base de données

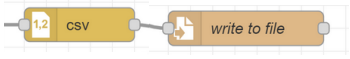
Node-RED permet d'enregistrer les données dans un fichier texte ou dans une base de données. Il suffit d'utiliser le node approprié.

Fichier CSV:

La node **CSV** permet de formater une chaine de caractères "string" en CSV à partir d'un objet JSON.



Il faut ajouter un node **write to file** pour enregistrer les lignes CSV dans un fichier.



108

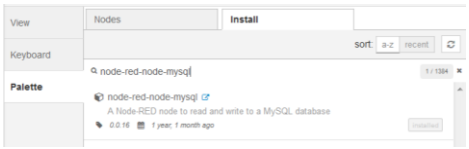
Node-RED

Node-RED Base de données

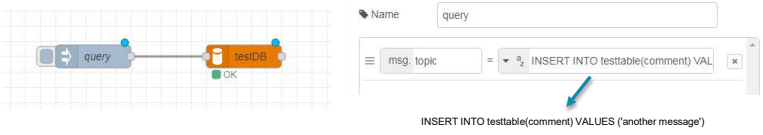
Base de données MySQL:

Un node de gestion du classique système de gestion de bases de données "SGBD" MySQL est disponible. Pour l'utiliser, il suffit d'installer le node mysql.

N.B: Cela sous-entend que vous avez MySQL installé et démarré avec une base de données accessible.



La requête sql est injectée au node **mysql** sous la forme d'un objet. C'est une chaine de caractères qui constitue la propriété topic de l'objet.



109

Chapitre 6

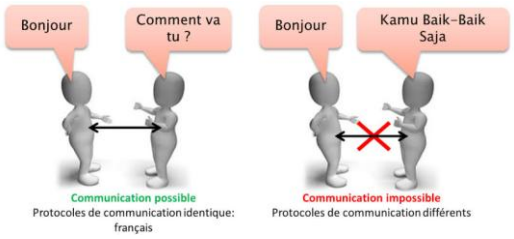
Les protocoles de communication pour l'IoT

110

Les protocoles de communication pour l'IoT

Les protocoles de communication

- Les protocoles de communication sont un ensemble de règles qui permettent aux appareils de communiquer entre eux.
- Les protocoles définissent la syntaxe, la sémantique et la synchronisation de la communication.
- Une analogie étroite avec les protocoles est celle des langues humaines.

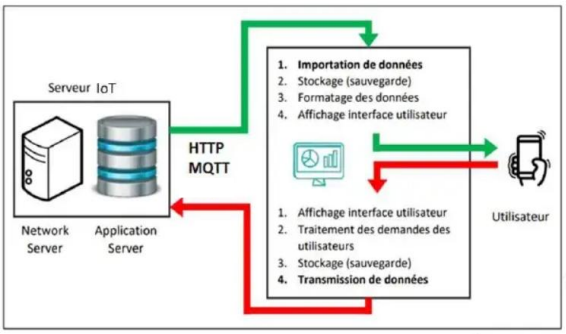


111

Les protocoles de communication pour l'IoT

Les protocoles de communication

- Il existe de nombreux protocoles de communication disponibles pour les applications IoT.
- Les protocoles suivants sont couramment utilisés : HTTP et MQTT.



112

Les protocoles de communication pour l'IoT

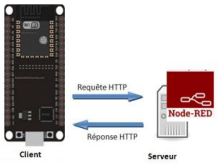
HTTP (Hypertext Transfer Protocol)

- Le protocole de transfert hypertexte (**HTTP**) est le protocole de communication derrière le World Wide Web (WWW).
- Il est basé sur une architecture client-serveur et fonctionne à la manière des demandes et des réponses.
- HTTP utilise TCP (protocole de contrôle de transmission) pour fournir des connexions fiables.

Pour HTTPS (Hypertext Transfer Protocol Secure) , les données envoyées à travers ce protocole sont cryptées par SSL (Secure Sockets Layer)/TLS (Transport Layer Security).

La communication entre le navigateur et le serveur se fait comme suit:

- ❑ Le navigateur (client) effectue une **requête** HTTP
- ❑ Le serveur traite la requête puis envoie une **réponse** HTTP



113

Les protocoles de communication pour l'IoT

Requête HTTP

- Une requête HTTP est un ensemble de lignes envoyées au serveur par le navigateur.
- Une ligne de requête : comprend trois éléments devant être séparés par un espace :
  - ❑ La méthode (GET, POST, DELETE, PUT,…)
  - ❑ L'URL
  - ❑ La version du protocole utilisé par le client (HTTP/1.0)

❑ Méthodes de la requête:

- GET : demande pour récupérer des données.
- POST : envoie de données.
- PUT: mettre à jour les données.
- DELETE : suppression des données

Format de la requête

Envoyé par le client au serveur

<Méthode> <URI> HTTP/<Version>
[<Champ d 'entête> : <Valeur>]
[<tab><Suite Valeur si >1024>]
ligne blanche
[corps de la requête pour la méthode POST]

```
GET /docu2.html HTTP/1.0
Accept: www/source
Accept: text/html
Accept: image/gif
User-Agent: Lynx/2.2 libwww/2.14
From: alice@pays.merveilles.net
* une ligne blanche *
```

```
POST /script HTTP/1.0
Accept: www/source
Accept: text/html
Accept: image/gif
User-Agent: Lynx/2.2 libwww/2.14
From: alice@pays.merveilles.net
Content-Length: 24
* une ligne blanche *
name1=value1&
name2=value2
```

GET
Retrieves data from the server
POST
Submit data to the server
PUT
Update data already on the server
DELETE
Deletes data from the server

114

Les protocoles de communication pour l'IoT

Réponse HTTP

- Une réponse HTTP est un ensemble de lignes envoyées au navigateur par le serveur.
- Une ligne de statut : précisant la version du protocole utilisé et l'état du traitement de la requête à l'aide d'un code et d'un texte explicatif. La ligne comprend trois éléments devant être séparés par un espace :
  - ❑ La version du protocole utilisé
  - ❑ Le code de statut
  - ❑ La signification du code

❑ Le code de statut:

- 1XX: Information
- 2XX: Success
- 3XX: Redirection
- 4XX: Client Error
- 5XX: Server Error

200 - OK
201 - OK created
301 - Moved to new URL
304 - Not modified (Cached version)
400 - Bad request
401 - Unauthorized
404 - Not found
500 - Internal server error

Format de la réponse

Réponse envoyé par le serveur au client
HTTP/<Version> <Status> <Commentaire Status>
Content-Type: <Type MIME du contenu>
[< Champ d 'entête >: <Valeur>]
[<tab><Suite Valeur si >1024>]
Ligne blanche
Document

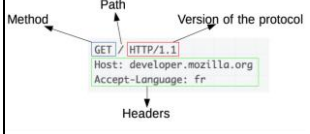


Les protocoles de communication pour l'IoT

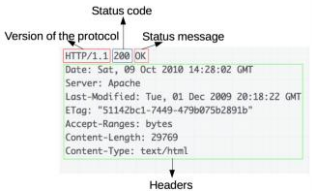
En-têtes HTTP

- Les headers sont les en-têtes HTTP pour les requêtes et les réponses.
- Les champs d'en-têtes sont un ensemble de lignes facultatives permettant de donner des informations supplémentaires.
- Chacune de ces lignes est composée d'un nom qualifiant le type d'en-tête, suivi de deux points (:) et de la valeur de l'en-tête.

Requête



Réponse

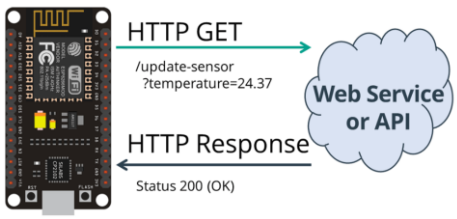


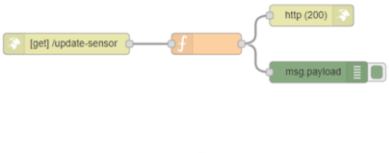
3 headers types:

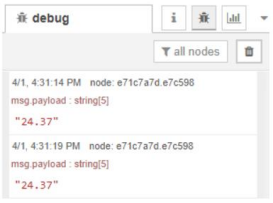
General:	Response:	Request:
Request URL	Server	Cookies
Request Method	Set-Cookie	Accept-xxx
Status Code	Content-Type	Content-Type
Remote Address	Content-Length	Content-Length
Referrer Policy	Date	Authorization
		User-Agent
		Referrer

Les protocoles de communication pour l'IoT

Exemple

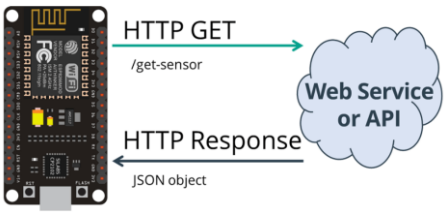


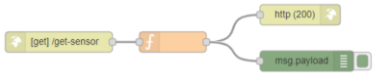


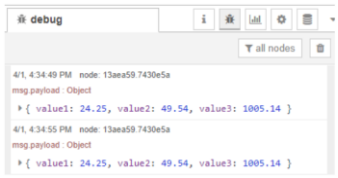


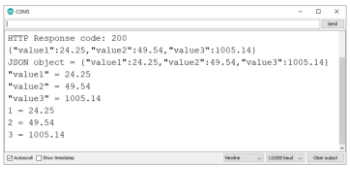
Les protocoles de communication pour l'IoT

Exemple







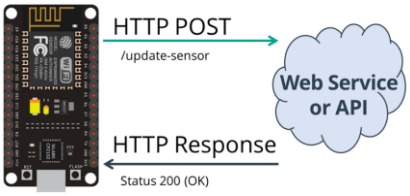


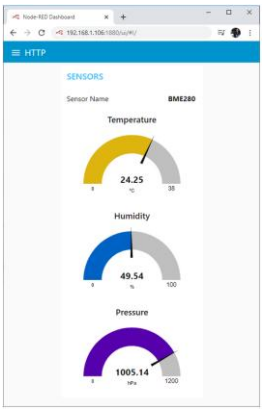
118

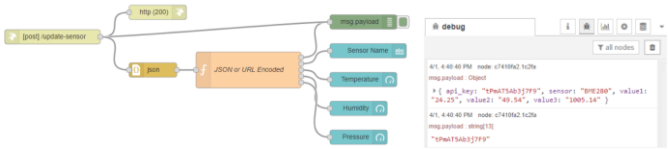
Les protocoles de communication pour l'IoT

Exemple









119

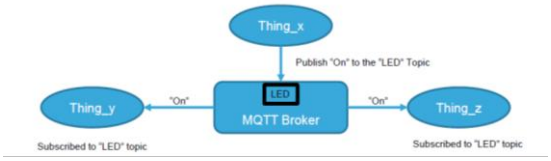
Les protocoles de communication pour l'IoT

MQTT

- MQTT est un protocole de communication machine à machine (M2M) utilisé dans l'IoT.
- MQTT est basé sur un système de publication et d'abonnement (publish/subscribe), où publish pour publier des données sur un serveur (également appelé broker), et le subscribe pour s'abonner au serveur et recevoir des données du serveur.
- Le broker MQTT est responsable de la distribution des messages et peut-être quelque part dans le Cloud.

- MQTT utilise un modèle client/serveur dans lequel chaque appareil IoT est un client et est connecté à MQTT broker.

- Les clients envoient des messages à une adresse appelée topic. Le broker MQTT transmettra ce message à tous les clients abonnés à ce topic.

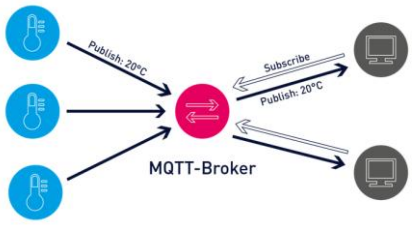


120

Les protocoles de communication pour l'IoT

Broker MQTT

- Le broker (courtier/intermédiaire) est principalement chargé de recevoir tous les messages, de filtrer les messages, de décider qui les intéresse et de publier le message à tous les clients abonnés.
- On peut utiliser un **broker local** comme « Mosquitto broker » (géré par la fondation eclipse) qui peut être **installé sur une carte** comme Raspberry Pi, ou un MQTT **broker sur le cloud**.

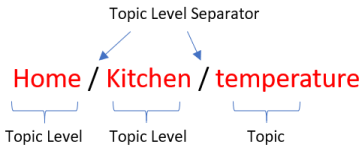


121

Les protocoles de communication pour l'IoT

Topic

- Le sujet (Topic): c'est l'endroit vers ou depuis lequel un appareil souhaite placer ou récupérer un message.
- Le topic fait référence à une chaîne de caractères que le broker utilise pour filtrer les messages pour chaque client connecté.
- Les topics sont représentés avec des chaînes séparées par une barre oblique. Chaque barre oblique indique un niveau de topic.
- Exemple : **Home/kitchen/temperature**



122

Les protocoles de communication pour l'IoT

Topic

- Les topics sont sensibles à la casse, ce qui rend le sujet « maison/bureau/temperature » différent de « maison/bureau/**T**emperature » .
- Un client peut s'abonner à des branches entières de l'arborescence à l'aide des deux caractères génériques suivants **(wildcards) :**
  - ❑ **le signe plus (+)**, qui correspond à un seul niveau.

Un client peut recevoir toutes les températures: maison/+/temperature
Il recevra donc les températures de la cuisine (maison/cuisine/temperature), du salon (maison/salon/temperature) et du bureau (maison/bureau/temperature) … .

  - ❑ **le dièse (#)** pour plusieurs niveaux.

Par exemple, si un client s'abonne au topic « maison/salon/# », il recevra toutes les données du salon. De même, s'il s'abonne au topic « maison/# », il collectera toutes les données des capteurs déployés dans la maison (température, humidité, ...).





123

Les protocoles de communication pour l'IoT

Autre composants

- **Les messages:** sont les informations échangées entre les appareils. Que ce soit une commande ou des données.

- **Publish/Subscribe**: le modèle publish/subscribe découple le client qui envoie un message (publisher) du ou des clients qui reçoivent les messages (subscribers).
  Les publishers et les subscribers ne se contactent jamais directement. En fait, ils ne sont même pas conscients que l'autre existe. La connexion entre eux est gérée par un troisième composant: le broker.

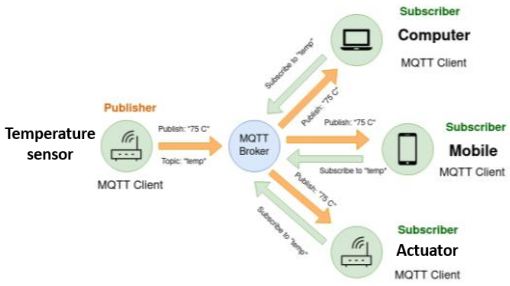
  - Publish: le processus qu'un appareil utilise pour envoyer son message au broker.
  - Subscribe: est le processus qu'un appareil utilise pour récupérer un message du broker.

Les protocoles de communication pour l'IoT

Exemple

Le **publisher** dans ce cas est un capteur de température qui envoie les informations de température au broker MQTT.

Il y a trois **subscribers** (abonnés):
- Mobile,
- Un tableau de bord Web sur Computer,
- Un actionneur qui contrôle une autre machine.

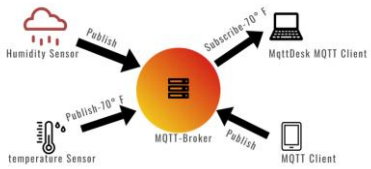


Lorsque le capteur envoie ses informations au topic « temp », tous les abonnés à ce topic reçoivent une copie de ce message.

Les protocoles de communication pour l'IoT

Architecture « publish/subscribe »

- Le protocole MQTT utilise une architecture « publish/subscribe » tandis que le protocole HTTP utilise une architecture « request/response ».
- Contrairement au principe de client/serveur utilisé sur le Web, MQTT utilise celui de la publication/souscription. En effet, plusieurs clients se connectent au broker soit pour publier des informations, soit pour souscrire à leur réception.
- La différence est importante, car elle évite de devoir demander (requête) des données dont on ne sait pas quand elles arriveront.
- Une donnée sera directement transmise aux Subscribers dès qu'elle aura été reçue dans le Broker. Pour recevoir les données appartenant à un Topic, le Subscriber doit d'abord souscrire à un Topic.





Les protocoles de communication pour l'IoT

Qualité de service (QoS)

MQTT intègre la qualité de service (QoS). En effet, le publisher a la possibilité de définir la qualité de son message.

Trois niveaux sont possibles :

- **Au plus une fois: At most once: QoS=0:** le plus rapide, mais le moins fiable.
- **Au moins une fois: At least once: QoS=1**
- **Exactement une fois: Exactly once: QoS=2 :** le plus lent, mais le plus fiable.

MQTT Message	Description
CONNECT	Client request to connect to server
CONNACK	Connect acknowledgement
PUBLISH	Publish message
PUBACK	Publish acknowledgement
PUBREC	Publish received
PUBREL	Publish release
PUBCOMP	Publish complete
SUBSCRIBE	Client subscribe request
SUBACK	Subscribe acknowledgement
UNSUBSCRIBE	Unsubscribe request
UNSUBACK	Unsubscribe acknowledgement
PINGREQ	PING request
PINGRESP	PING response
DISCONNECT	Client is disconnecting

127

Les protocoles de communication pour l'IoT

At most once QoS=0

- Un message de QoS niveau 0 « At most once ». Signifie que le message est envoyé sans garantie de réception (le broker n'informe pas l'expéditeur qu'il a reçu le message).
  - Le destinataire n'accuse pas la réception du message, et le message n'est pas stocké et retransmis par l'expéditeur. Le message peut être perdu si le client est déconnecté ou si le serveur échoue.
  - QoS0 est le mode de transfert le plus rapide et souvent appelé « fire and forget »



PUBLISH	Publish message

128

Les protocoles de communication pour l'IoT

At least once QoS=1

- Un message de QoS niveau 1 « At least once ». Le client transmettra plusieurs fois s'il le faut jusqu'à ce que le Broker lui confirme que le message était transmis sur le réseau.
  - QoS=1 est le mode de transfert par défaut.
  - QoS=1 garantit qu'un message est remis au moins une fois au destinataire.
  - L'expéditeur stocke le message jusqu'à ce qu'il reçoive un paquet PUBACK (acknowledgment) du destinataire qui accuse la réception du message.
  - Si l'expéditeur ne reçoit pas de PUBACK, le message est envoyé à nouveau avec le flag DUP (duplicate) activé jusqu'à ce qu'un accusé de réception soit reçu.
  - Il est alors possible qu'un message soit envoyé ou remis plusieurs fois.
  - Le message doit être stocké localement chez l'expéditeur et le destinataire jusqu'à ce qu'il soit traité.
  - Le message est supprimé du destinataire après son traitement.



PUBLISH	Publish message
PUBACK	Publish acknowledgement

129

Les protocoles de communication pour l'IoT

Exactly once QoS=2

- Un message de QoS niveau 2 « exactly once » sera obligatoirement sauvegardé par l'émetteur et le transmettra toujours tant que le récepteur ne confirme pas son envoi sur le réseau.

  - QoS=2 est le plus haut niveau de service dans MQTT. Ce niveau garantit que chaque message n'est reçu qu'une seule fois par les destinataires prévus.
  - Le message doit être stocké localement chez l'expéditeur et le destinataire jusqu'à ce qu'il soit traité.
  - QoS 2 est le niveau de QoS le plus sûr et le plus lent.
  - Ce niveau délivre le message avec deux **paires de paquets**.
  - Le message peut être traité au niveau du récepteur après la première transmission.
  - L'expéditeur et le destinataire utilisent l'identifiant de paquet du message PUBLISH d'origine pour coordonner la livraison du message.



PUBLISH	Publish message
PUBREC	Publish received
PUBREL	Publish release
PUBCOMP	Publish complete

130

Les protocoles de communication pour l'IoT

Exactly once QoS=2

- Dans la première paire de transmission :
  L'expéditeur transmet le message et obtient un accusé de réception du destinataire qu'il a **stocké le message**.
  Si l'expéditeur ne reçoit pas d'accusé de réception, le message est à nouveau envoyé avec le drapeau DUP activé jusqu'à ce qu'un accusé de réception soit reçu.

- Dans la deuxième paire de transmission,:
  L'expéditeur indique au destinataire qu'il peut **terminer le traitement** avec un message PUBREL.
  Si l'expéditeur ne reçoit pas l'accusé de réception du message PUBREL, le message PUBREL est renvoyé jusqu'à ce qu'un accusé de réception soit reçu.
  L'expéditeur supprime le message qu'il a enregistré lorsqu'il reçoit l'accusé de réception du message PUBREL.

Le récepteur peut traiter le message dans la première ou la deuxième phase.
Le destinataire renvoie un message d'achèvement à l'expéditeur indiquant qu'il a **terminé de traiter** le message (PUBCOMP).

131

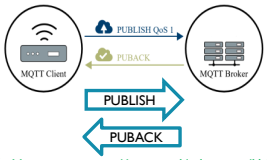
Les protocoles de communication pour l'IoT

QoS0 « Au plus une fois »



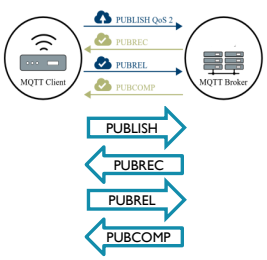
✗ Les messages sont supprimés après l'envoi
✗ Les messages ne sont pas acquittés

QoS1 « Au moins une fois »



✓ Messages sauvegardés tant qu'il n'y a pas d'ACK
✓ Une perte de connexion peut être résolue
✗ Un message peut être reçu/envoyé plusieurs fois

QoS2 « Exactement une fois »

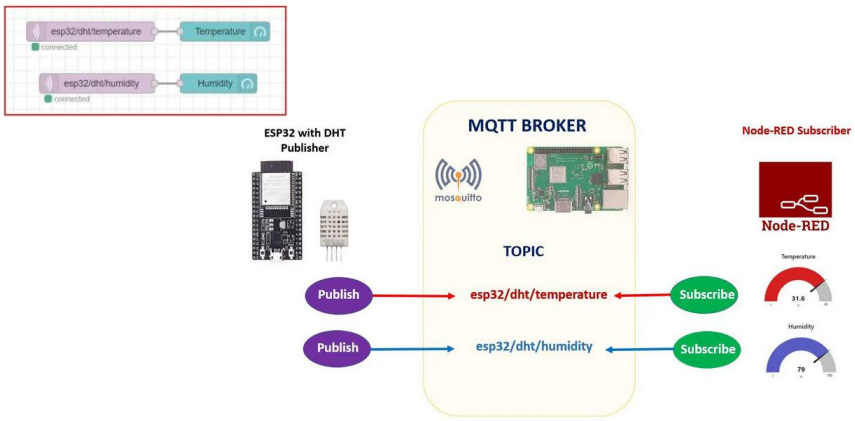


✓ Messages sauvegardés tant qu'il n'y a pas d'ACK
✓ Une perte de connexion peut être résolue
✓ Le message ne sera reçu/envoyé qu'une seule fois

132

Les protocoles de communication pour l'IoT

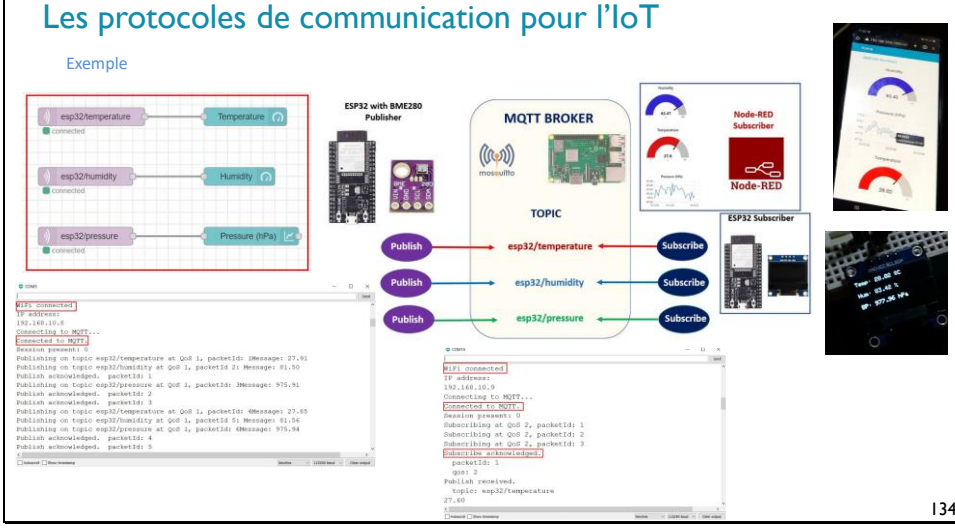
Exemple



133

Les protocoles de communication pour l'IoT

Exemple



134

Chapitre 7

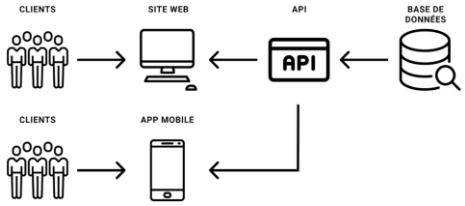
REST API

135

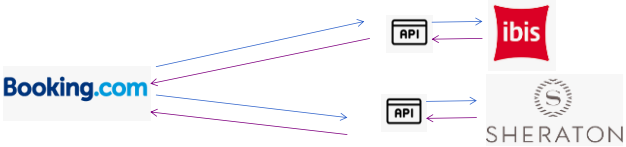
REST API

API Concept

- L'interface de programmation d'applications (API) présente des fonctions et des règles qui permettent l'interaction et la communication entre différentes applications.



Exemple : Rechercher un hôtel (pays, arrivée, départ, nombre de personnes, nombre de chambres) ---→--- Search

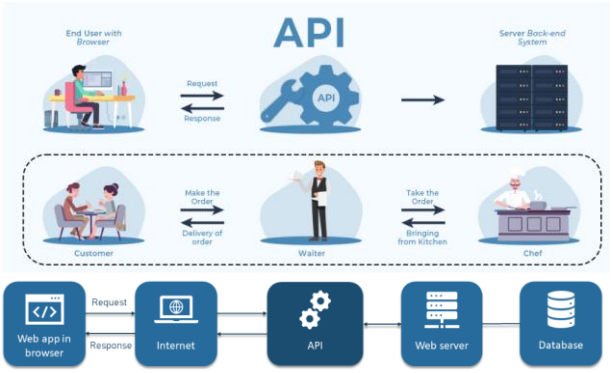


136

REST API

API Concept

- L'API assure la médiation entre les applications via des requêtes et des réponses. Elle a donc un double rôle :
  - ✓ Elle transmet la requête du client au serveur.
  - ✓ Elle transmet la réponse du serveur au client.



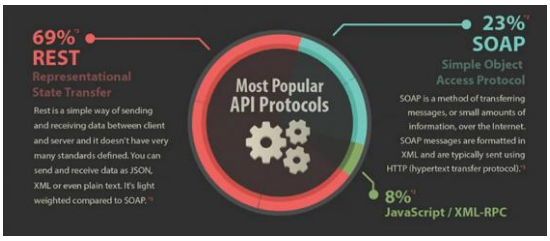
137

REST API

Type d'architecture API

L'API utilise divers protocoles et architectures pour envoyer des requêtes et des réponses :

- ❑ **RPC (Remote Procedure Call).**
- ❑ **SOAP (Simple Object Access Protocol).**
- ❑ **REST (Representational State Transfer).**



138

REST API

RPC (Remote Procedure Call)

- **RPC (Remote Procedure Call)** : le client effectue un appel de fonction à distance (également appelée méthode ou procédure) sur un serveur. Généralement, une ou plusieurs valeurs de données sont transmises au serveur lors de l'appel.
- Les styles RPC les plus utilisés sont XML-RPC et JSON-RPC.

  - **XML-RPC** utilise XML pour décrire les réponses/requêtes et les protocoles HTTP pour le transfert d'informations du client au serveur.

  - **JSON-RPC** est un RPC léger similaire à XML. Ici, le protocole est encodé en JSON. Le format JSON est privilégié en raison de sa lisibilité et de sa flexibilité.



RPC

```
POST /SendUserMessage HTTP/1.1
Host: api.example.com
Content-type: application/json

{"userId": 501, "message": "Hello!"}
```

- RPC se concentre sur les fonctions ou les actions.

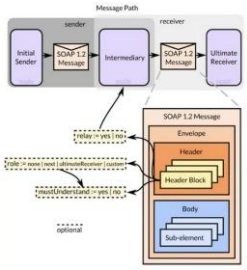
REST API

SOAP (Simple Object Access Protocol)

- **SOAP (Simple Object Access Protocol)** : un protocole standard initialement conçu pour que des applications développées avec différents langages sur différentes plateformes puissent communiquer des messages. Il permet aux clients d'appeler des services Web et de recevoir des réponses.
- SOAP utilise des protocoles intégrés qui en rendent l'utilisation complexe.
- Il définit la méthode de communication et la structure des messages.
- Ses échanges utilisent des formats de fichier XML.
- Comme ce protocole traite des messages, l'objectif principal est d'empêcher tout accès non autorisé grâce à la sécurité de WS (Web Services Security).

SOAP







REST API

REST (Representational State Transfer)

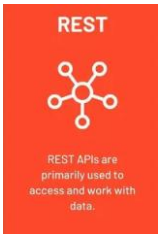
- **REST (Representational State Transfer)** : transfert d'état représentatif est **un style architectural** pour la conception d'API via le protocole HTTP. Son principal avantage est **sa grande flexibilité.**
- REST permet de créer des ressources (documents, images et fichiers) sur le World Wide Web.
- Le client REST demande au serveur d'effectuer une action sur une ressource serveur spécifique. Les actions sont limitées à la création, à la lecture, à la mise à jour et à la suppression (CRUD) uniquement et sont transmises sous forme de verbes HTTP ou de méthodes HTTP.



REST

```
#%RAML 1.0
title: ACME Banking API
version: 1.0

/customers:
  get:
  post:
  /{customer_id}:
    get:
    patch:
```



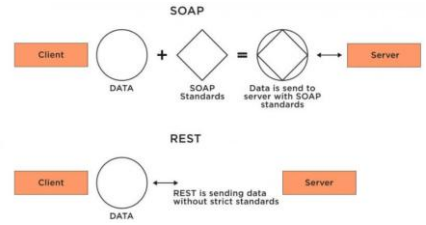
- REST se concentre sur les ressources ou les objets.

REST API

REST et SOAP

- REST et SOAP sont des approches différentes de la transmission des données en ligne. Plus précisément, toutes deux définissent la manière de développer des **API** qui permettent les échanges de données entre plusieurs applications Web.
    - REST est un ensemble de **principes architecturaux**.
    - SOAP est un protocole officiel **géré par le W3C (World Wide Web Consortium).**
- La principale différence entre les deux est que SOAP est un protocole, REST non.
- En général, les API suivent l'approche REST ou SOAP en fonction de leur utilisation et des préférences du développeur.

SOAP (Simple Object Access Protocol)	REST (REpresentational State Transfer)
Protocol	Architecture
Function driven	Data driven
Requires advanced security, but can define it, too	Relies on underlying network which can be less secure
Needs more bandwidth	Only needs minimum bandwidth
Stricter rules to follow	Easier for developers to suggest recommendations
Cannot use REST	Can use SOAP
Only works in XML	Works in different data formats such as HTML, JSON, XML, and plain text
Supports HTTP and SMTP protocols	Only requires HTTP



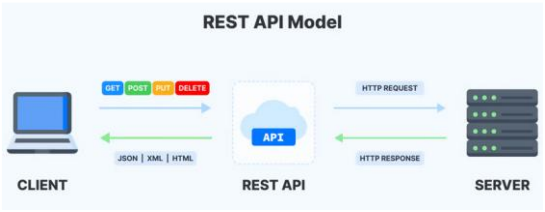
142

REST API

REST API principe

- Une **REST API** est une interface de programmation d'application qui suit les principes de l'**architecture REST**.
- Elle permet à différentes applications de communiquer entre elles via des **requêtes HTTP** standard, facilitant ainsi l'**échange de données** et la **gestion des opérations**.

➔ En adoptant ces principes, une **REST API** permet aux développeurs de **créer des applications Web** et **mobiles flexibles**, interopérables et évolutives. Elle favorise également la réutilisation des fonctionnalités, car différentes applications peuvent communiquer efficacement en utilisant des standards ouverts et bien connus.



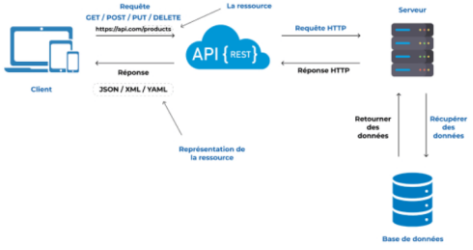
143

REST API

Composants de REST API

- ❑ **Client :** un client ou un programme lancé du côté de l'utilisateur (sur son appareil) initiant la communication.
- ❑ **Serveur :** un serveur utilisant des API, comme accès à ses fonctions et données.
- ❑ **Ressource :** tout contenu que le serveur transmet au client.

Le **client** envoie une requête HTTP en précisant **la ressource**, **le serveur** traite la requête en récupérant les informations demandées dans sa base de données et ensuite renvoie une représentation de la ressource.



144

REST API

Caractéristiques clés de REST API

- **Ressources** : Une REST API expose des ressources, qui sont des entités identifiables, telles que des objets, des données ou des services. Chaque ressource est identifiée par une URI (Uniform Resource Identifier), ce qui permet de la localiser facilement.

L'URI identifie une ressource et la différencie des autres en utilisant un nom, un emplacement ou les deux.
L'URL identifie l'adresse Web ou l'emplacement d'une ressource unique.



URIs

```
https://example.com/api/v3/products
https://example.com/api/v3/users
```

- Les ressources doivent être regroupées par nom et non par verbe. Une API pour obtenir tous les produits devrait être :

Correct

```
https://example.com/api/v3/products
```

```
https://example.com/api/v3/getAllProducts
```

Wrong

145

REST API

Caractéristiques clés de REST API

- **Verbes HTTP** : les opérations sur les ressources sont effectuées en utilisant les verbes HTTP standard tels que GET (récupérer), POST (créer), PUT (mettre à jour) et DELETE (supprimer). Chaque verbe a une signification spécifique dans l'interaction avec les ressources.





146

REST API

Caractéristiques clés de REST API

- **Représentations** : les ressources peuvent être représentées dans différents formats, tels que JSON (JavaScript Object Notation), XML (eXtensible Markup Language), HTML, etc.

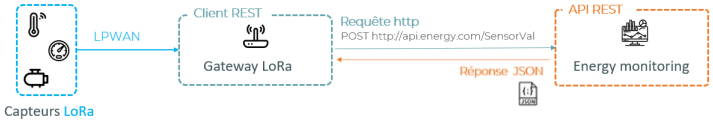




147

REST API

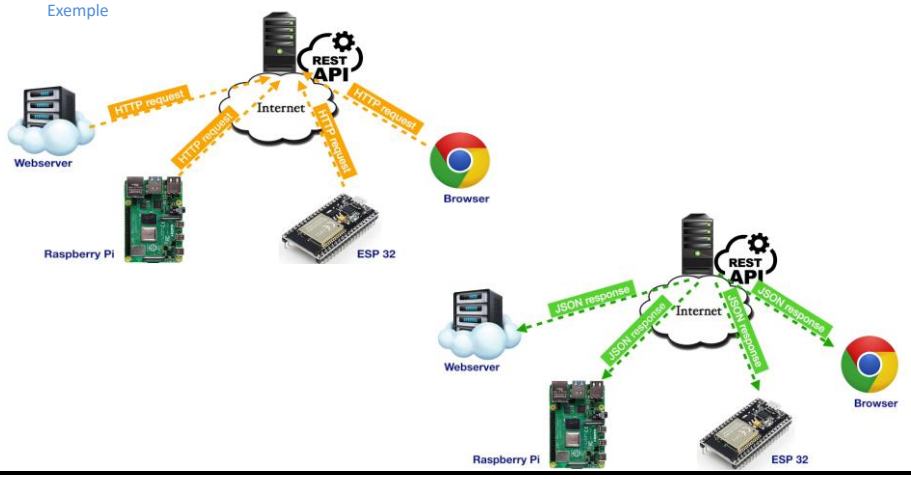
Solution IoT et REST API



148

REST API

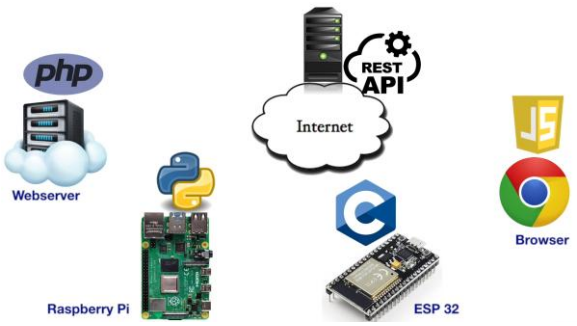
Exemple



149

REST API

Solution IoT et REST API



Rest API on Server and your client application can be written in different languages

They both can run on different OS and different types of hardware

150

REST API

Solution IoT et REST API



151

REST API

Solution IoT et REST API



152

REST API

Solution IoT et REST API



153

REST API

RPC vs SOAP vs REST



	RPC	SOAP	REST
Organized in terms of	local procedure calling	enveloped message structure	compliance with six architectural constraints
Format	JSON, XML, Protobuf, Thrift, FlatBuffers	XML only	XML, JSON, HTML, plain text,
Learning curve	Easy	Difficult	Easy
Community	Large	Small	Large
Use cases	Command and action-oriented APIs; internal high performance communication in massive micro-services systems	Payment gateways, identity management CRM solutions financial and telecommunication services, legacy system support	Public APIs simple resource-driven apps

154

Bibliographie

- Bhavana B. C., Vathsala Gowda C. T., & Rakshitha B. H. A Survey: **Internet of Things (IoT) Technologies, Applications**. IJRASET, 2022. doi : https://doi.org/10.22214/ijraset.2022.42595
- **Arduino Documentation Arduino Platform & Embedded Programming**, lien
- **ESP32 Technical Reference Manual**, lien
- Documentation officielle IBM, **l'architecture et les API de IBM Watson IoT**. IBM training Course Guide IoT Cloud Developer 2021.pdf
- **AWS IoT Core Developer Guide**, lien
- Tyagi, H., Kumar, R. (2020). **Cloud Computing for IoT.** In: Alam, M., Shakil, K., Khan, S. (eds) Internet of Things (IoT). Springer, Cham. https://doi.org/10.1007/978-3-030-37468-6_2
- **Node-RED**, Documentation, lien
- **MQTT Version 5.0**, lien
- B. Wukkadada, K. Wankhede, R. Nambiar and A. Nair, "**Comparison with HTTP and MQTT In Internet of Things (IoT)**," 2018 International Conference on Inventive Research in Computing Applications (ICIRCA), Coimbatore, India, 2018, pp. 249-253, doi: 10.1109/ICIRCA.2018.8597401.
- Richardson, L., Amundsen, M. and Ruby, S., 2013. **RESTful web APIs: services for a changing world**. " O'Reilly Media, Inc.", lien

155